CAPSLOCK: CONTAINER-AWARE POLICY SECURITY LOCK

A Kubernetes-Native Security Gateway for Environment-Aware Policy Enforcement, Risk-Based Deployment Validation, Adaptive Content Inspection, and Environment-Sensitive Load Balancing

A Final Year Group Research Dissertation

Submitted in partial fulfillment of the requirements for the

Bachelor of Science (Hons) in Information Technology

Component	Author	IT Number
MEDS — Multi-Environment Deployment System	Kulatunga K K M D	IT22347626
EAPE — Environment-Aware Policy Engine	Raigambandarage N K	IT22338716
SDLB — Service Discovery and Load Balancing	Pandukabhaya H. D. T.	IT22345028
ICAP Operator	S. A. L. Guruge	IT22353634

Project ID: 25-26J-043

Department of Computer Systems Engineering

Faculty of Computing

Sri Lanka Institute of Information Technology

April 2026

DECLARATION

We declare that this group dissertation is our own work and does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning. To the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

We hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this dissertation, in whole or in part in print, electronic or other medium. We retain the right to use this content in whole or part in future works.

Name	Student ID	Signature
Kulatunga K K M D	IT22347626	
Raigambandarage N K	IT22338716	
Pandukabhaya H. D. T.	IT22345028	
S. A. L. Guruge	IT22353634	

Signature of Supervisor: _____ Date: _____

Name of Supervisor: _____

DEDICATION

To our families and friends, whose patience and encouragement made this research possible.

To the open-source communities behind Kubernetes, Istio, ClamAV, OPA, and Kyverno — whose publicly documented engineering made this research both tractable and reproducible.

To Mr. Amila Senarathne and Mr. Kavinga Yapa Abeywardena — for the direction, the trust, and the standards you held us to.

ACKNOWLEDGEMENTS

The authors wish to express sincere gratitude to the supervisor, Mr. Amila Senarathne, and co-supervisor, Mr. Kavinga Yapa Abeywardena, at the Sri Lanka Institute of Information Technology for their continuous guidance, constructive criticism, and invaluable technical feedback throughout this research project. Their domain expertise in container security and distributed systems was instrumental in shaping the design decisions underpinning the CAPSLock platform.

Gratitude is extended to the Department of Computer Systems Engineering at SLIIT, whose provision of laboratory infrastructure, access to academic databases, and supportive faculty made this research possible. The feedback obtained from the research panel during progress presentations contributed significantly to the refinement of the research problems and the sharpening of the evaluation methodology.

Finally, the authors acknowledge the maintainers of the open-source technologies upon which this work is built — Kubernetes, Istio, FastAPI, ClamAV, Prometheus, OPA Gatekeeper, and Kyverno — whose collective engineering effort forms the foundation upon which applied security research such as the present study is made practical.

ABSTRACT

Contemporary Kubernetes deployments face a fundamental tension between operational velocity and security rigour. The delivery lifecycle demands rapid promotion of workloads through development, staging, and production environments, yet the security tooling available to platform engineering teams was designed for static, perimeter-based threat models that predate container orchestration. GitOps tools automate the mechanics of deployment without performing quantified risk assessment. Admission control frameworks enforce static policy sets with no awareness of environment context. Content inspection services operate outside the orchestration layer, disconnected from scheduling and scaling decisions. Load balancing policies apply uniformly across all workload tiers.

CAPSLock is a four-component Kubernetes-native security gateway addressing these deficiencies. MEDS provides deployment-time enforcement through an eight-step promotion pipeline with six-factor risk scoring and the novel Unified Security Lifecycle Orchestration (USLO) model. EAPE provides automated namespace environment classification through a seven-factor confidence scoring algorithm, applying CIS Benchmark v1.9 and PCI-DSS v4.0 compliant policies through a seven-step automated pipeline. SDLB dynamically generates tier-appropriate Istio routing configurations with an adaptive decision engine. The Kubernetes ICAP Operator automates ClamAV-backed scanning pod lifecycle through a declarative CRD, with a composite health score driving reactive and predictive autoscaling.

Empirical evaluation confirms that MEDS achieves 100% test pass rates across state-machine enforcement, risk scoring, ICAP gate, and audit chain integrity. EAPE achieves 73% overhead reduction with 100% production compliance coverage and 163 ms end-to-end pipeline latency. SDLB achieves 0.7% Production distribution variance and 7.99% routing overhead. The ICAP Operator achieves 94.7% malware detection true-positive rate versus 78.2% for manual baseline.

Keywords: Kubernetes, CAPSLock, MEDS, EAPE, SDLB, ICAP Operator, Policy-as-Code, OPA Gatekeeper, Kyverno, CIS Benchmarks, PCI-DSS, ClamAV, Istio

TABLE OF CONTENTS

Contents

DECLARATION 2
DEDICATION 3
ACKNOWLEDGEMENTS 4
ABSTRACT 5
TABLE OF CONTENTS 6
LIST OF TABLES 9
LIST OF ABBREVIATIONS 9
1. INTRODUCTION 11
1.1. Background and Context 11
1.2. The CAPSLock Research Programme 12
1.3. Research Problems and Gaps 13
1.4. Research Objectives 14
1.5. Thesis Structure 15
2. LITERATURE REVIEW 16
2.1. Kubernetes Security and Misconfiguration 16
2.2. GitOps and Deployment Pipeline Security 16
2.3. Policy-as-Code and Admission Control 17
2.4. ICAP Protocol and Content Inspection 17
2.5. Service Mesh and Environment-Aware Routing 18
2.6. Autoscaling and Security-Operational Metrics 18
2.7. Compliance Frameworks 19
2.8. Summary and Research Positioning 19
3. SYSTEM ARCHITECTURE 20
3.1. Overall CAPSLock Architecture 20
3.2. Component Integration Contracts 21
3.3. Data Flow Through the Pipeline 21
3.4. Technology Stack 22
4. METHODOLOGY 23
4.1. Research Approach 23
4.2. MEDS: Eight-Step Promotion Pipeline and USLO Design 23
4.2.1. Research Problem and Design Rationale 23
4.2.2. The Eight-Step Promotion Pipeline 24
4.2.3. USLO Mode Specifications 26
4.2.4. Promotion Decision Threshold Design 27

4.2.5. SHA-256 Chained Audit Log 27
4.2.6. LLM Natural Language Interface 27
4.2.7. Audit Chain and Snapshot Design 28
4.3. EAPE: Seven-Factor Detection and Policy Pipeline Design 28
4.3.1. Design Rationale 28
4.3.2. The Seven-Factor Detection Algorithm 28
4.3.3. The Seven-Step Policy Application Pipeline 30
4.3.4. Policy Tiers and Inheritance 31
4.3.5. Advanced Features 31
4.3.6. CIS Benchmark v1.9 Compliance Catalogue Design 31
4.3.7. PCI-DSS v4.0 Compliance Catalogue Design 32
4.4. SDLB: Environment-Aware Routing and Adaptive Decision Engine Design 33
4.4.1. Design Rationale 33
4.4.2. Three-Tier Routing Policies 33
4.4.3. Adaptive Decision Engine 34
4.4.4. Routing Decision State Machine 35
4.4.5. Compliance Audit Logging 35
4.4.6. Service Discovery Mechanism 35
4.4.7. Inspection Traffic Flow 36
4.5. ICAP Operator: CRD Design, Health Scoring, and Autoscaling 36
4.5.1. Design Rationale 36
4.5.2. ICAPService Custom Resource Definition 37
4.5.3. Composite Health Scoring Formula 37
4.5.4. Dual Autoscaling Architecture 38
4.5.5. Signature Update Strategy 38
4.5.6. Health Score Normalisation and Computation 38
4.5.7. Reconciliation Loop and Operator Robustness 39
4.5.8. NIST SP 800-190 Compliance Alignment 40
5. IMPLEMENTATION 41
5.1. MEDS Implementation 41
5.2. EAPE Implementation 42
5.3. SDLB Implementation 43
5.4. ICAP Operator Implementation 44
6. EVALUATION AND RESULTS 46
6.1. Evaluation Overview and Methodology 46
6.2. MEDS: Promotion Pipeline Validation 47
6.2.1. Experimental Setup 47

6.2.2. State-Machine Enforcement Results 47
6.2.3. Risk-Scoring Determinism 47
6.2.4. ICAP Gate Behaviour 48
6.2.5. Audit Chain Integrity 48
6.3. EAPE: Detection, Policy, and Compliance Results 50
6.3.1. Environment Detection Results 50
6.3.2. Policy Pipeline Performance 50
6.3.3. Policy Selection, Translation, and Compliance 51
6.4. SDLB: Routing Correctness, Failover, and Overhead 52
6.4.1. Routing Policy Correctness 52
6.4.2. Failover and Recovery 53
6.4.3. Overhead and Scalability 53
6.5. ICAP Operator: Health Scoring and Autoscaling Results 54
6.5.1. Health Score Baseline and Scenario Evaluation 54
6.5.2. Autoscaling Responsiveness 56
6.5.3. Malware Detection Accuracy 56
6.5.4. Consolidated Benchmark Summary 56
6.6. Cross-Component Consolidated Results 57
6.7. Experimental Environment Summary 59
6.8. Cross-Component Consolidated Results 59
7. CONCLUSION 62
7.1. Summary of Contributions 62
7.2. Research Findings 63
7.3. Limitations 65
7.3.1. Evaluation Environment 65
7.3.2. Weight Calibration 65
7.3.3. Compliance Catalogue Scope 66
7.3.4. Scale and Longitudinal Validity 66
7.4. Future Work 66
7.4.1. Cross-Platform Validation 66
7.4.2. Machine-Learning-Based Optimisation 67
7.4.3. Extended Compliance Frameworks 67
7.4.4. Continuous Integration and Platform Integration 67
7.4.5. Longitudinal Production Study 68
References 69

LIST OF TABLES

Table 1: CAPSLock Inter-Component Integration Contracts 21
Table 2: CAPSLock Technology Stack 22
Table 3: MEDS Six-Factor Risk Model — Weights and Signal Sources 25
Table 4: USLO Mode Specifications Across Environment Tiers 26
Table 5: MEDS Promotion Decision Thresholds 27
Table 6: EAPE Seven-Factor Detection Algorithm — Factors and Weights 29
Table 7:EAPE Policy Tiers with Enforcement and ICAP Mode 31
Table 8: CIS Benchmark v1.9 Control Distribution Across Sections 32
Table 9: PCI-DSS v4.0 Requirement Group Distribution 33
Table 10: ICAPService CRD Domain Structure 37
Table 11: ICAP Operator Health Score Metric Weights 37
Table 12: ICAP Operator Health Score Normalisation Formulas 39
Table 13: ICAP Operator NIST SP 800-190 Compliance Mappings 40
Table 14: MEDS Module Responsibilities 42
Table 15: EAPE Package and Module Responsibilities 43
Table 16: SDLB Component Responsibilities 44
Table 17: ICAP Operator Controller Responsibilities 45
Table 18: MEDS Risk Score Examples Across Promotion Scenarios 48
Table 19: MEDS Evaluation Results Summary 49
Table 20: EAPE Environment Detection Results 50
Table 21: EAPE Performance Benchmark Results 51
Table 22: EAPE Evaluation Results Summary 51
Table 23: SDLB Routing Policy Correctness Results 52
Table 24: SDLB Failover and Recovery Results 53
Table 25: ICAP Operator Health Score Across Seven Controlled Scenarios 55
Table 26: ICAP Operator Evaluation Results Summary 55
Table 27: ICAP Operator Consolidated Benchmark Summary 56
Table 28: CAPSLock Platform Consolidated Results Across All Components 58
Table 29: Experimental Environment Specifications per Component 59
Table 30: CAPSLock Cross-Component Consolidated Results 60

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
CAPSLock	Container-Aware Policy Security Lock
CIS	Centre for Internet Security
ClamAV	Clam AntiVirus (open-source antivirus engine)
CNCF	Cloud Native Computing Foundation
CRD	Custom Resource Definition
EAPE	Environment-Aware Policy Engine (Component 2)
GitOps	Git-based Operations (declarative deployment practice)
HPA	Horizontal Pod Autoscaler
ICAP	Internet Content Adaptation Protocol (RFC 3507)
JSONL	JSON Lines (newline-delimited JSON)
K8s	Kubernetes
LLM	Large Language Model
MEDS	Multi-Environment Deployment System (Component 1)
mTLS	Mutual Transport Layer Security
NIST	National Institute of Standards and Technology
OPA	Open Policy Agent
PCI-DSS	Payment Card Industry Data Security Standard
REST	Representational State Transfer
SDLB	Service Discovery and Load Balancing (Component 3)
SHA	Secure Hash Algorithm
SLIIT	Sri Lanka Institute of Information Technology
TLS	Transport Layer Security
USLO	Unified Security Lifecycle Orchestration
YAML	YAML Ain't Markup Language

1. INTRODUCTION

1.1. Background and Context

The containerisation of software workloads and the subsequent emergence of Kubernetes as the dominant orchestration platform have transformed the security landscape of cloud-native application delivery in ways that established security architecture was not designed to address. Where traditional security models relied on stable network perimeters, long-lived host identities, and relatively infrequent software deployments, containerised workloads are characterised by ephemerality, elastic scaling, dynamic service discovery, and continuous delivery pipelines that may promote hundreds of changes per day. The security tooling designed for the former model cannot be mechanically applied to the latter without introducing significant coverage gaps.

Three such gaps are consistently documented in the empirical literature. Rahman et al.[1] catalogued 1,051 security misconfigurations across 2,039 open-source Kubernetes manifests, finding that practitioners agreed to remediate only 60% of reported issues. Shamim et al.[2] identified 11 security practices requiring consistent application across environment tiers and concluded that no single existing tool addresses their full breadth. Sultan et al.[3] confirmed that content-borne threats remain structurally unaddressed by infrastructure-level policy controls alone. These findings, taken together, characterise the state of Kubernetes security practice as fragmented across tools, inconsistent across environment tiers, and lacking integration between the content inspection layer and the policy enforcement layer.

Alongside these empirical findings, two architectural features of modern Kubernetes deployments amplify the risk. First, the widespread adoption of GitOps tooling — ArgoCD, FluxCD, Spinnaker — has automated the mechanical execution of deployment but has not introduced any mechanism for enforcing security-policy governance, quantifying deployment risk, or integrating content inspection into the promotion decision. A promotion that removes a network policy or introduces a privileged container may pass all automated quality gates and reach production without any security-aware gate having been consulted. Second, Kubernetes's flat-by-

default network model, documented by Minna et al., creates lateral movement risk that cannot be addressed by admission control alone, and ICAP-based content inspection, standardised in RFC 3507, remains disconnected from orchestration despite being a mature and widely deployed content security mechanism.

The CAPSLock research project was motivated by the need for a cohesive platform that closes all three gaps simultaneously, as a coherent security gateway rather than as a collection of independent tools. The four components of CAPSLock address the deployment promotion gap (MEDS), the environment-contextual policy enforcement gap (EAPE), the environment-aware routing gap (SDLB), and the managed content inspection gap (ICAP Operator), through a shared data model, defined integration contracts, and a unified compliance framework alignment.

1.2. The CAPSLock Research Programme

CAPSLock (Container-Aware Policy Security Lock) is a group research project developed by four members of the Faculty of Computing at the Sri Lanka Institute of Information Technology, Project ID 25-26J-043. The platform is designed as a Kubernetes-native security gateway that intercepts, validates, and routes workload traffic through a multi-layer security enforcement pipeline. The four components and their principal contributions are summarised as follows.

The Multi-Environment Deployment System (MEDS), developed by Kulatunga K K M D (IT22347626), is the orchestration core through which every environment promotion must pass. MEDS introduces the Unified Security Lifecycle Orchestration (USLO) model — a novel framework for governing the evolution of security policies across environment tiers — and a weighted six-factor risk model that quantifies deployment risk prior to promotion. Every promotion traverses an eight-step pipeline before reaching its target environment.

The Environment-Aware Policy Engine (EAPE), developed by Kaavya [Last Name] (IT22338716), provides automated namespace environment classification and compliance-aligned policy enforcement. EAPE contributes a seven-factor confidence scoring algorithm for environment detection, a five-dimension policy conflict detection mechanism with three configurable resolution strategies, and a dual-engine

translation system that generates enforcement artefacts for both OPA Gatekeeper and Kyverno from a shared compliance catalogue covering CIS Kubernetes Benchmarks v1.9 and PCI-DSS v4.0.

The Service Discovery and Load Balancing component (SDLB), developed by Pandukabhaya H. D. T. (IT22345028), provides environment-aware routing for ICAP inspection traffic within the cluster. SDLB consumes environment classification signals from EAPE and dynamically generates Istio VirtualService and DestinationRule configurations appropriate to each environment tier, supplemented by an adaptive decision engine that responds to traffic hysteresis, health-penalty-weighted effective-rate scoring, and predictive spread-mode transitions.

The Kubernetes ICAP Operator, developed by S. A. L. Guruge (IT22353634), automates the full lifecycle of ClamAV-backed ICAP content inspection pods through a declarative Custom Resource Definition. The operator contributes a composite health scoring formula derived from four security-operational metrics and drives both reactive autoscaling through the Kubernetes HPA and predictive autoscaling through a trend-based pre-emptive scaling path.

1.3. Research Problems and Gaps

The four research problems addressed by this project share a common structural characteristic: each represents a point at which the existing Kubernetes tooling ecosystem provides no integrated solution, and at which the absence of a solution creates measurable security risk.

The deployment promotion gap arises from the absence of security-aware gating in GitOps workflows. Contemporary GitOps tooling accepts a merged pull request as sufficient authorisation for promotion, with no mechanism for quantifying the security-posture implications of the change, enforcing content-scanning obligations, or validating policy evolution against environment-specific compliance thresholds. The consequence is that promotions which should be blocked routinely reach production.

The environment-contextual policy enforcement gap arises from the static nature of existing admission control frameworks. OPA Gatekeeper and Kyverno enforce the policies they are given, but they have no concept of environment tier and no mechanism for automatically adapting enforcement to the deployment context of the namespace they govern. Teams are compelled to maintain separate, manually curated policy catalogues for each tier, producing the systematic neglect of encryption and audit controls documented by Verdet et al.

The environment-aware routing gap arises from the uniform application of service discovery and load balancing policies across all workload tiers. A routing configuration calibrated for production inspection strictness imposes unnecessary overhead in development; a configuration calibrated for development leaves production infrastructure exposed to undetected endpoint failures and unbalanced load. No existing Kubernetes networking component provides environment-contextual routing policy selection for ICAP inspection traffic.

The managed content inspection gap arises from the absence of a Kubernetes-native lifecycle management mechanism for ICAP and ClamAV services. ClamAV scanning pods are typically deployed manually and managed outside the orchestration layer, without automated signature updates, health-aware autoscaling, or security-metric-driven scaling decisions.

1.4. Research Objectives

The overall objective of the CAPSLock project is to design, implement, and empirically evaluate a cohesive four-component Kubernetes security gateway that closes the deployment promotion, policy enforcement, routing, and content inspection gaps identified in Section 1.3. The component-specific objectives are as follows.

MEDS objectives: design and implement an eight-step promotion pipeline with state-machine transition enforcement and environment-specific decision thresholds; propose and implement the USLO model for managing security policy evolution across environment tiers; develop a six-factor weighted risk scoring model; implement a three-layer ICAP scanning architecture with fallback; and provide tamper-evident audit logging through a cryptographically chained audit log.

EAPE objectives: design and implement a seven-factor confidence scoring algorithm for namespace environment detection producing a confidence value in [0, 1]; develop a seven-step automated policy application pipeline incorporating compliance validation against CIS Benchmark v1.9 and PCI-DSS v4.0; implement five-dimension conflict detection with three configurable resolution strategies; and generate enforcement artefacts for both OPA Gatekeeper and Kyverno from a shared compliance catalogue.

SDLB objectives: design and implement an environment-aware routing controller that generates tier-appropriate Istio configurations from EAPE classification signals; develop an adaptive decision engine with traffic hysteresis detection, health-penalty-weighted effective-rate scoring, and predictive failover; and validate routing correctness, failover behaviour, and overhead against defined performance thresholds.

ICAP Operator objectives: design and implement a Kubernetes-native operator with a declarative ICAPService CRD for full ICAP and ClamAV lifecycle management; develop a composite health scoring formula from four security-operational metrics; implement both reactive HPA-based and predictive autoscaling paths; and demonstrate improved malware detection accuracy relative to manual and CPU-only autoscaling baselines.

1.5. Thesis Structure

This thesis is structured as a combined research document presenting the findings of all four CAPSLock components. Chapter 2 reviews the literature relevant to all four components, covering Kubernetes security, GitOps, admission control, ICAP, service mesh networking, autoscaling, and compliance frameworks. Chapter 3 presents the overall CAPSLock system architecture, component integration contracts, data flow, and technology stack decisions. Chapter 4 presents the methodology for all four components in depth, covering design rationale, algorithmic decisions, and detailed specifications. Chapter 5 describes the implementation of each component. Chapter 6 presents the evaluation results for each component. Chapter 7 concludes with a consolidated statement of contributions, research findings, limitations, and future work directions.

2. LITERATURE REVIEW

2.1. Kubernetes Security and Misconfiguration

Rahman et al.[1] provide the most comprehensive empirical characterisation of Kubernetes security misconfiguration, analysing 2,039 open-source manifests and cataloguing 1,051 defects across 11 categories. The finding that practitioners agreed to remediate only 60% of reported issues is the most operationally significant result: it confirms that manual misconfiguration remediation cannot close the security gap at scale. Shamim et al.[2] synthesised 11 security commandments from 104 practitioner resources and confirmed that no single tool addresses all 11. Minna et al.[4] demonstrated that Kubernetes's flat network model creates lateral movement risk that admission controllers cannot independently address, and that NetworkPolicy density is a reliable per-namespace security posture indicator. Cesarano and Natella[5] showed that RBAC cannot constrain field-level Pod specification values, establishing the technical necessity of webhook-level admission control as a complement to RBAC. Sultan et al.[3] confirmed that content-borne threats remain unaddressed by infrastructure-level policy controls, establishing the complementary relationship between admission enforcement and content inspection.

2.2. GitOps and Deployment Pipeline Security

GitOps tooling — ArgoCD, FluxCD, Spinnaker — has substantially improved the reproducibility and auditability of Kubernetes deployments by treating version-controlled repositories as the authoritative source of cluster state. However, as Humble and Farley established in their foundational work on continuous delivery, automation of the deployment mechanism does not of itself provide any security guarantee: a mechanically correct deployment of a policy-violating workload is still a security incident. Contemporary GitOps tooling provides no mechanism for quantifying deployment risk, enforcing security policy evolution governance, integrating content-scanning obligations, or validating compliance posture prior to promotion. These absences define the research space that MEDS addresses.

Published incident analyses consistently cite configuration drift, removed security controls, and the absence of pre-promotion validation as primary root causes of

container security incidents. The MEDS USLO model is motivated by the observation that policy drift between development and production environments is not merely a theoretical risk but a documented and recurring pattern in production Kubernetes deployments.

2.3. Policy-as-Code and Admission Control

OPA Gatekeeper[6] implements the Open Policy Agent as a Kubernetes validating admission webhook using the Rego declarative language. Its principal strength is the expressive power of Rego, which supports cross-resource reasoning and can encode complex compliance requirements with high fidelity. Kyverno[7] provides a more accessible authoring model through native Kubernetes ClusterPolicy resources using CEL expressions. Sissodiya et al.[8] formalised the semantic differences between the two engines through SMT-based verification, demonstrating that they produce distinct enforcement outcomes for semantically equivalent policy intent — providing the academic justification for EAPE's dual-engine approach. Verdet et al.[9] analysed 812 open-source Terraform repositories and found that access policy controls are consistently adopted while encryption and audit controls are routinely neglected, motivating the compliance-first policy selection in EAPE's catalogue. Rice[10] establishes that security policy enforcement must operate at multiple independent layers simultaneously and cannot be reduced to perimeter controls, directly informing EAPE's multi-layer compliance framework alignment.

2.4. ICAP Protocol and Content Inspection

The Internet Content Adaptation Protocol, standardised in IETF RFC 3507[11], defines a lightweight HTTP-like protocol for offloading content processing from proxy servers to dedicated adaptation services. Two interaction modes are defined: REQMOD for request-side adaptation (antivirus scanning, URL filtering) and RESPMOD for response-side adaptation (DLP, content transformation). Ilca and Lucian demonstrated practical ICAP integration with ClamAV for enterprise environments, confirming that the c-icap open-source server provides a viable bridge between the ICAP protocol and the ClamAV signature engine. Despite broad deployment across enterprise proxy products, no prior published work places ICAP and ClamAV within the Kubernetes

orchestration model as managed, lifecycle-aware resources. The MEDS three-layer ICAP scanning architecture and the ICAP Operator address this gap from complementary angles: MEDS integrates ICAP scanning into the promotion decision pipeline, while the ICAP Operator manages the lifecycle of the ICAP services that MEDS and SDLB consume.

2.5. Service Mesh and Environment-Aware Routing

Li et al.[12] survey service mesh architectures, identifying service discovery, traffic management, observability, and security as the four primary problem domains. Saleh Sedghpour et al.[13] provide empirical evidence that Istio traffic management policies require per-environment calibration — production-optimised routing introduces unacceptable latency overhead in development, and development-tolerant routing leaves production infrastructure exposed. NIST SP 800-204A[14] endorses Kubernetes with Istio as the reference platform for secure microservices architecture, recommending non-bypassable enforcement through the Envoy sidecar proxy. Hahn et al.[15] evaluated security vulnerabilities in Consul, Istio, and Linkerd under default and optimal configurations, confirming that service mesh security requires additional admission-level enforcement to close gaps in the data plane. These findings collectively justify SDLB's design: a controller that generates environment-appropriate Istio configurations from policy engine signals rather than relying on a single static configuration.

2.6. Autoscaling and Security-Operational Metrics

Nguyen et al.[16] provide the foundational analysis of Kubernetes HPA limitations, demonstrating that CPU and memory metrics fail to capture application-specific demand for workloads whose primary resource consumption is not CPU-bound. For a ClamAV-backed ICAP service, the operationally critical signals — scan latency, error rate, backlog depth, and signature freshness — are entirely absent from the default HPA model. Xu et al. provide the most comprehensive empirical study of bugs in Kubernetes operators, identifying incorrect reconciliation loops, unsafe concurrent access, and missing finalizer logic as recurring patterns. Their findings motivated the defensive design choices in the ICAP Operator: server-side apply for idempotent

writes, explicit finalizer management, and reconciliation rate limiting. Burns et al.[17] established the desired-state reconciliation model as a fundamental Kubernetes design principle, tracing its lineage to Google's Borg and Omega systems.

2.7. Compliance Frameworks

The CIS Kubernetes Benchmarks v1.9[18] define 28 controls across five sections (RBAC, Pod Security, Network Policy, Secrets Management, Namespace Isolation) that are directly applicable to namespace-level admission enforcement. PCI-DSS v4.0[19] imposes 16 Kubernetes-applicable requirements across four groups (stored data protection, transmission security, secure systems, audit logging) for environments processing payment card data. NIST SP 800-190[20] provides the normative risk taxonomy bridging compliance requirements and Kubernetes-native controls across five primary risk areas. All three frameworks are referenced throughout EAPE's compliance catalogue and SDLB's compliance audit logging, ensuring that the CAPSLock platform's enforcement decisions have traceable regulatory justification.

2.8. Summary and Research Positioning

The literature establishes four structural gaps that no existing single tool or combination of tools addresses cohesively: the absence of security-aware deployment gating in GitOps workflows; the static, context-unaware nature of admission control frameworks; the uniform application of routing policies without environment awareness; and the absence of Kubernetes-native lifecycle management for ICAP content inspection services. CAPSLock addresses all four through a shared architectural model in which MEDS, EAPE, SDLB, and the ICAP Operator are designed as components of a cohesive pipeline rather than as independent tools, with defined integration contracts and a shared compliance framework alignment.

3. SYSTEM ARCHITECTURE

3.1. Overall CAPSLock Architecture

CAPSLock is deployed to a dedicated capslock-system namespace within a Kubernetes cluster, with each of the four components running as a Kubernetes Deployment. The components communicate through a combination of REST API calls and Kubernetes-native event mechanisms — namespace label watches, Custom Resource Definition updates, and standard Kubernetes Service endpoints — following the desired-state reconciliation model established by Burns et al. as the fundamental Kubernetes architectural principle.

The high-level data flow is as follows. A developer submits a deployment promotion through the MEDS REST API. MEDS executes the eight-step promotion pipeline, which includes a call to EAPE to retrieve the compliance posture of the target namespace. EAPE classifies the namespace environment and applies appropriate admission control policies. SDLB watches for EAPE classification events and updates Istio routing configurations accordingly. The ICAP Operator manages the scanning pods that MEDS contacts for content inspection and that SDLB routes inspection traffic to. The management dashboard, served by EAPE, provides a unified operational view of all four components' state.

The system is built on the following shared infrastructure: K3s as the lightweight Kubernetes distribution for development and evaluation; OPA Gatekeeper v3.14 and Kyverno v1.11 as the dual admission enforcement engines; Istio as the service mesh for SDLB traffic management; Prometheus for metrics collection across all components; ClamAV backed by c-icap as the scanning engine managed by the ICAP Operator; and a shared mTLS configuration for inter-component communication.

3.2. Component Integration Contracts

The four components share a set of integration contracts that define the API surface consumed by each component. These contracts were designed to be minimally coupled — each component depends only on the endpoints it needs and exposes only what it is responsible for — while remaining sufficient for the full CAPSLock pipeline to function.

Caller	Endpoint	Data Exchanged
MEDS	POST /api/policies/apply (EAPE)	Namespace ID → PolicyApplicationResult: environment, compliance score, violations, remediation
SDLB	GET /api/icap/health (EAPE)	Aggregate ICAP health score for routing weight decisions
ICAP Operator	GET /api/integration/icap/policy-status/{ns} (EAPE)	Per-namespace policy approval gate before activating scanning
SDLB	ICAPService CRD status (ICAP Operator)	Per-instance health score (0–100) for penalty-weighted routing
MEDS	ICAP scanning endpoint (ICAP Operator pods)	REQMOD content scan request / scan result response

Table 1: CAPSLock Inter-Component Integration Contracts

3.3. Data Flow Through the Pipeline

A complete promotion event traverses all four components in a defined sequence. When MEDS receives a promotion request, it first validates the state-machine transition (confirming that the source and target environments form a permitted pair), then contacts the ICAP Operator scanning pods for content inspection, then queries EAPE for the compliance posture of the target namespace. EAPE's response includes the environment classification, the compliance score, and any violations with remediation guidance. MEDS incorporates the compliance posture into its six-factor risk score, applies the USLO engine to plan policy migration, and makes the promotion

decision against environment-specific thresholds. If the decision is APPROVE, MEDS snapshots the policy state and delegates to the GitOps orchestrator.

Concurrently, EAPE's classification of the namespace triggers SDLB to update the Istio VirtualService and DestinationRule for the namespace, ensuring that inspection traffic routed to the namespace is governed by a routing policy appropriate to its environment tier. SDLB's routing decisions are informed by the per-instance health scores published by the ICAP Operator to its ICAPService CRD status subresource, with health-degraded instances receiving reduced routing weight through the penalty-weighted effective-rate scoring mechanism.

3.4. Technology Stack

Component	Technologies	Language / Runtime
MEDS	FastAPI, Kubernetes client, ClamAV/ICAP, SHA-256 audit chain, LLM integration	Python 3.11
EAPE	controller-runtime, OPA Gatekeeper, Kyverno, YAML policy templates	Go 1.21 binary + Python 3.11 FastAPI bridge
SDLB	Kubernetes controller-runtime, Istio API client, Prometheus metrics	Go (controller) + Python (metrics bridge)
ICAP Operator	Kubebuilder, controller-runtime, Kubernetes HPA, c-icap, ClamAV	Go 1.21
Shared	K3s v1.29, Minikube v1.29 (eval), OPA Gatekeeper v3.14, Kyverno v1.11, Istio, Prometheus	—

Table 2: CAPSLock Technology Stack

All components are packaged as multi-stage Docker containers, following CIS Benchmark container image security guidance. Go binaries are built in a builder stage and copied into minimal runtime images, reducing the container attack surface. All inter-component API calls are secured with mutual TLS, configurable via environment variables. Kubernetes RBAC is configured with least-privilege service accounts for each component, implementing the CIS Section 4.1 RBAC controls that EAPE enforces for managed namespaces.

4. METHODOLOGY

4.1. Research Approach

All four CAPSLock components were developed following the constructive research paradigm, in which the primary contribution is an artefact evaluated against defined performance criteria. Each component was developed iteratively, with design decisions validated through implementation and testing before the next design decision was made. This methodology was preferred over a waterfall approach because the integration dependencies between components required working implementations to validate API contracts, and the empirical nature of several algorithmic choices — notably the EAPE detection weights and the MEDS risk factor weights — required calibration against real data.

The four components were developed in parallel by four independent researchers, with weekly integration sessions to validate inter-component contracts. Each component has its own test suite and evaluation methodology, described in the component-specific sections below, with system-level integration testing performed jointly across all four components in the final development phase.

4.2. MEDS: Eight-Step Promotion Pipeline and USLO Design

4.2.1. Research Problem and Design Rationale

MEDS is positioned as the deployment-time security-enforcement middleware layer through which every environment promotion must pass. Its design rationale is that deployment promotion is a security-critical operation that carries three distinct obligations: the content of the artefact being promoted must be inspected; the quantitative security-posture change introduced by the promotion must be assessed; and the evolution of security policies implied by the promotion must be governed according to the target environment's compliance requirements. No existing GitOps tool addresses any of these three obligations, and the design of MEDS is structured around providing all three as a cohesive pipeline.

4.2.2. The Eight-Step Promotion Pipeline

Every promotion request submitted to MEDS traverses the following eight steps in sequence. The pipeline is non-bypassable: no step may be skipped, and failure at any step halts the promotion.

Step 1 — State-Machine Transition Enforcement: MEDS maintains a directed state machine over the three environment tiers (development, staging, production). Valid transitions are development→staging and staging→production; no direct development→production promotion is permitted. The state machine is enforced before any subsequent step is invoked, ensuring that promotions cannot skip the staging gate.

Step 2 — Three-Layer ICAP Content Scanning: The promoted artefact is scanned through a three-layer architecture with fallback. In cluster environments with a deployed ICAP Operator, scanning is performed through the operator-managed ClamAV pods via the ICAP REQMOD protocol. In local development environments, scanning is performed through a policy-engine gateway. In offline CI environments, scanning is simulated deterministically. The three-layer architecture ensures that content inspection is never silently skipped due to infrastructure unavailability.

Step 3 — Compliance Posture Retrieval: MEDS calls EAPE's POST /api/policies/apply endpoint for the target namespace, receiving the environment classification, the compliance score, and any active violations with remediation guidance. This step embeds EAPE's full seven-step policy pipeline into the promotion workflow: the promotion decision is informed by EAPE's real-time compliance assessment.

Step 4 — Six-Factor Risk Scoring: MEDS computes a quantitative risk score across six weighted factors: configuration complexity delta (the magnitude of the configuration change), policy change volume and direction (how many policies are being added or removed, and whether additions reduce security), version delta (the number of revisions since the last promotion), environment-transition validity (whether the transition crosses a security boundary), ICAP coverage score (whether content inspection was fully executed), and compliance posture score (from EAPE's

response). Each factor is normalised to a 0–100 scale and combined into a composite score through the weighted formula.

Risk Factor	Weight	Signal Source
Configuration complexity delta	0.20	Change set analysis
Policy change volume and direction	0.20	Policy diff against baseline
Version delta since last promotion	0.15	Git version history
Environment-transition validity	0.20	State-machine gate result
ICAP content scanning coverage	0.10	Scanning layer response
Compliance posture score	0.15	EAPE PolicyApplicationResult

Table 3: MEDS Six-Factor Risk Model — Weights and Signal Sources

Step 5 — USLO Policy Planning: The USLO engine evaluates the policy changes implied by the promotion against the target environment's governance rules and produces a policy-migration plan that either approves, conditions, or blocks the changes.

Step 6 — Promotion Decision: The composite risk score is compared against environment-specific thresholds (< 60 for development-to-staging, < 40 for staging-to-production). If the threshold is exceeded, MEDS returns a structured rejection containing the risk factor breakdown, contributing EAPE violations, and remediation guidance.

Step 7 — Policy-Version Snapshotting: An immutable snapshot of the namespace security state is created on every approved promotion, enabling rollback to any prior configuration.

Step 8 — GitOps Deployment: The promotion is delegated to the GitOps orchestrator with a SHA-256 chained audit log entry providing cryptographic proof of the decision.

4.2.3. USLO Mode Specifications

The USLO engine defines four operational modes governing how security policy changes are evaluated for each target environment tier.

USLO Mode	Environment	Policy Additions	Policy Removals
Permissive	Development	Accepted in audit-only mode; no justification required	Accepted without justification
Controlled	Staging	Require CIS or PCI-DSS compliance mapping	Require documented justification and risk acceptance
Enforced	Production	Require compliance mapping and security review approval	Require executive approval and compensating control documentation
Locked	Prod (PCI scope)	Require executive and compliance officer approval	Prohibited except under emergency change procedure

Table 4: USLO Mode Specifications Across Environment Tiers

The transition from Controlled to Enforced is the most significant governance boundary in the USLO model. Policy additions that pass the staging gate must independently justify their compliance basis when targeting production, because PCI-DSS cardholder data environment requirements may be more stringent than those applicable to staging. In Locked mode, every policy removal is treated as an emergency change requiring an out-of-band change ticket reference validated against the organisation's change management system.

4.2.4. Promotion Decision Threshold Design

Transition	Risk Threshold	Rationale
Development to Staging	< 60	Permissive; staging is controlled and not externally exposed
Staging to Production	< 40	Strict; production may be PCI-DSS scoped; risk elevation is significant
Emergency Promotion (any)	< 80 with executive override	Auditable path for urgent patches that would normally be blocked

Table 5: MEDS Promotion Decision Thresholds

The threshold asymmetry is deliberate: a promotion scoring between 40 and 60 should be used as an opportunity to investigate and reduce risk before production promotion is attempted. The emergency promotion path provides an auditable channel for critical security patches while still requiring a verifiable executive override token, ensuring no silent threshold bypass is possible.

4.2.5. SHA-256 Chained Audit Log

The audit log is implemented as a SHA-256 chained structure. Each log entry contains the event data (promotion identifier, timestamp, risk score, decision, policy snapshot reference), a SHA-256 hash of that data, and the SHA-256 hash of the previous entry. Any retrospective modification to any entry invalidates all subsequent entries' chain hashes, enabling detection of field-level tampering. This structure satisfies PCI-DSS Requirement 10 without requiring a separate database system.

4.2.6. LLM Natural Language Interface

MEDS integrates with a configurable large-language-model inference provider to supply an operator-facing natural language interface. Operators may query the system in plain English — asking why a promotion was rejected, what compliance violations are blocking a deployment, or what remediation steps are required — and receive

contextually relevant responses generated from the promotion record and EAPE's violation details. This feature reduces the cognitive burden of interpreting structured JSON compliance reports, particularly for operators who are not specialists in the referenced compliance frameworks.

4.2.7. Audit Chain and Snapshot Design

The SHA-256 chained audit log provides tamper-evident logging without a separate database. Each entry contains the event data, a SHA-256 hash of that data, and the hash of the previous entry. Retrospective modification to any entry invalidates all subsequent chain hashes, satisfying PCI-DSS Requirement 10's tamper-evidence mandate. The policy-version snapshot captures four elements for each approved promotion: the EAPE PolicyApplicationResult, the Istio VirtualService and DestinationRule configuration, the ICAPService CRD specification, and the active Gatekeeper Constraints and Kyverno ClusterPolicies. Rollback restores all four simultaneously and traverses the MEDS pipeline as a synthetic reverse promotion, allowing USLO to validate consistency before execution.

4.3. EAPE: Seven-Factor Detection and Policy Pipeline Design

4.3.1. Design Rationale

EAPE's design is structured around two principles drawn from the empirical literature. First, observable state takes precedence over declared intent: because namespace metadata labels are frequently absent or incorrect, EAPE derives environment context from observable structural properties of the namespace that remain informative even when explicit labelling is neglected. Second, compliance takes precedence over operational convenience: the policy selection algorithm prioritises CIS Benchmark and PCI-DSS requirements over operational policy considerations, ensuring that regulatory mandates are enforced consistently regardless of operational context.

4.3.2. The Seven-Factor Detection Algorithm

EAPE detects namespace environment tier using a weighted confidence scoring algorithm that produces both a classification (development, staging, or production) and a confidence value in [0, 1]. The confidence value enables MEDS to configure a

human-override threshold — for example, requiring administrator confirmation for namespaces classified with confidence below 0.50 — making the system's uncertainty explicit and actionable rather than silently resolved with a default.

Factor	Signal Source	Weight
Primary Kubernetes label (environment, env)	Label match on namespace	+0.60
Namespace name token (prod, staging, dev)	Name pattern match	+0.20
Security-level label	security-level or security label present	+0.20
PCI-DSS compliance label	compliance-pci-dss label present	+0.10
CIS compliance label	compliance-cis label present	+0.10 (combined cap +0.20)
Cluster characteristics	Node taints; cloud provider hints via ClusterDetector	variable
No-label name-only fallback	Namespace name only; no labels present	max 0.30

Table 6: EAPE Seven-Factor Detection Algorithm — Factors and Weights

The primary label factor carries the highest weight (0.60) because it represents an explicit, administratively authorised assertion. When present, it dominates the classification. When absent, detection falls through to the remaining factors. The 0.30 maximum confidence for the no-label name-only fallback correctly signals that namespaces with no labels at all are uncertain classifications, ensuring MEDS applies human oversight for such cases.

4.3.3. The Seven-Step Policy Application Pipeline

When MEDS calls POST /api/policies/apply, EAPE executes a seven-step pipeline:

- Step 1 — Environment Detection: The seven-factor algorithm is applied, producing an environment type and confidence score.
- Step 2 — Policy Selection: All loaded policy templates are scored against the detected environment context; the highest-scoring matching template is selected. This step executes in 882 ns/op — sub-microsecond.
- Step 3 — Conflict Detection: Pairwise comparison of candidate templates across five semantic dimensions: scanning mode (CRITICAL severity), environment scope (HIGH), compliance coverage (HIGH), resource limits (MEDIUM), and security level (LOW). Detection completes in 7.2 µs for a full template scan.
- Step 4 — Conflict Resolution: One of three configurable strategies is applied: precedence (production > staging > development), security-first (strictest scanning mode wins), or environment-aware (policy matching detected environment wins, falling back to precedence).
- Step 5 — Compliance Validation: CIS Benchmark v1.9 (28 controls, 5 sections) and PCI-DSS v4.0 (16 requirements, 4 groups) are evaluated against the namespace's live Kubernetes resources, producing per-control pass/fail results with remediation text.
- Step 6 — Status Reporting: A structured PolicyApplicationResult is returned to MEDS containing the environment, confidence score, selected policy, conflict resolutions, overall compliance status, per-framework scores, and violation objects with rule IDs, severities, and remediation guidance.
- Step 7 — Service Verification: EAPE confirms ICAP scanning service health via the SDLB health endpoint before marking the policy as applied.

4.3.4. Policy Tiers and Inheritance

EAPE ships three policy templates that inherit shared baseline controls from base-policy.yaml through a recursive deep-merge mechanism (LoadPolicyTemplateWithBase()). This inheritance ensures that no baseline security control is accidentally omitted when authoring a tier-specific template.

Template	Environment	Enforcement	ICAP Mode	Compliance
dev-policy	development	audit	log-only	CIS v1.9
staging-policy	staging	enforce	warn	CIS v1.9 + PCI-DSS v4.0
prod-policy	production	strict	block	CIS v1.9 + PCI-DSS v4.0

Table 7:EAPE Policy Tiers with Enforcement and ICAP Mode

4.3.5. Advanced Features

Three advanced features extend the core pipeline. Continuous Change Detection (Watch() method) polls namespace environment on a configurable interval and automatically re-applies the correct policy when a namespace's classification shifts — for example, when a namespace is relabelled from development to production during a migration — without requiring a new MEDS promotion. Policy Inheritance ensures no baseline control is ever omitted through the base-policy.yaml deep-merge. Conflict Audit Persistence appends every resolution event to a JSONL audit log served via GET /api/conflict-audit.

4.3.6. CIS Benchmark v1.9 Compliance Catalogue Design

The 28 CIS Benchmark v1.9 controls are implemented across five sections, each with its own Go source file in pkg/compliance/cis/. The distribution of controls across sections reflects the CIS consensus process's assessment of where the highest-value security configurations lie for namespace-level enforcement.

CIS Section	Controls	Primary Security Goal
4.1 RBAC Controls	6	Prevent privilege escalation through service accounts and ClusterRole abuse
4.2 Pod Security Controls	8	Enforce secure container runtime configurations (no privileged, non-root, read-only FS)
4.3 Network Policy Controls	4	Enforce namespace network isolation through default-deny ingress and egress
4.4 Secrets Management Controls	5	Prevent secrets exposure through environment variables and uncontrolled access
4.5 Namespace Isolation Controls	5	Enforce resource quotas, limit ranges, and label governance

Table 8: CIS Benchmark v1.9 Control Distribution Across Sections

Each control returns a ComplianceResult struct containing a pass/fail boolean, the violated field if applicable, the CIS control identifier (e.g. CIS-4.2.1 for the privileged container check), a severity rating from CRITICAL through LOW, and a remediation string specifying the exact Kubernetes manifest field that must be changed to pass the control. The remediation strings are authored to be directly usable by developers: for example, the CIS-4.2.1 remediation reads 'Set securityContext.privileged to false in the container specification' rather than simply referencing the control title.

4.3.7. PCI-DSS v4.0 Compliance Catalogue Design

The 16 PCI-DSS v4.0 requirements are implemented across four groups. The selection of these 16 requirements from the full 64-requirement standard was guided by the PCI Security Standards Council's container security supplement, which identifies the requirements most directly applicable to Kubernetes workload configurations.

PCI-DSS Group	Reqs.	Kubernetes Implementation
Req. Group 3 — Stored Data Protection	4	Etcd encryption-at-rest, Secrets RBAC scope, secret access audit logging
Req. Group 4 — Transmission Security	4	TLS-only NetworkPolicy enforcement, mTLS service configuration
Req. Group 6 — Secure Systems	4	Image digest pinning, pull policy enforcement, capability restrictions

PCI-DSS Group	Reqs.	Kubernetes Implementation
Req. Group 10 — Audit Logging	4	Kubernetes audit policy configuration, log retention, tamper detection

Table 9: PCI-DSS v4.0 Requirement Group Distribution

The most operationally significant PCI-DSS control in the EAPE catalogue is Requirement 6.3.2 (image digest pinning), which requires all container images to specify an explicit SHA256 digest rather than a mutable tag. This control prevents the silent drift of container content that occurs when a mutable tag such as latest is re-pointed to a different image, which is one of the primary vectors through which malicious code has historically entered production container environments. EAPE enforces this by denying Pods whose image references do not contain an @ sha256: digest specifier.

4.4. SDLB: Environment-Aware Routing and Adaptive Decision Engine Design

4.4.1. Design Rationale

SDLB addresses the routing uniformity problem identified in the research gap: a single Istio routing configuration cannot simultaneously satisfy the strictness requirements of production and the permissiveness requirements of development. The design principle is that routing policy should be a function of environment context, derived from EAPE's classification signals rather than manually configured per namespace. The adaptive decision engine extends this principle by ensuring that routing responds dynamically to infrastructure health, not merely to the static environment classification.

4.4.2. Three-Tier Routing Policies

SDLB defines three base routing policies, each expressed as an Istio VirtualService and DestinationRule configuration applied to the target namespace:

Production — Round-Robin with Health Gating: Strict round-robin load balancing across all ICAP instances that have passed active OPTIONS-based health verification. No traffic is routed to an instance with a health score below the configured health gate

threshold. This policy prioritises inspection coverage completeness and balanced utilisation.

Staging — Weighted 50/30/20 Routing: A weighted distribution across the three ICAP instances (50% to icap-a, 30% to icap-b, 20% to icap-c), enabling per-instance performance observation under load conditions approximating production without imposing the strict health gating of the production policy.

Development — Permissive Single-Instance Routing: Requests are routed to the first available healthy endpoint. Advisory-only health checks are applied — a health check failure generates a warning but does not remove the endpoint from rotation. This policy minimises routing overhead and inspection latency, supporting developer velocity.

4.4.3. Adaptive Decision Engine

The adaptive decision engine sits above the three base policies and modifies routing decisions in response to runtime signals. It introduces four mechanisms:

Traffic Spike Detection and Spread Mode: The engine monitors request rate against a configurable traffic spike threshold. When the rate exceeds the threshold, the engine transitions to spread mode, distributing load across all available healthy instances regardless of the base policy weights. A hysteresis cooldown window prevents oscillation between normal and spread modes.

ICAP Health Scoring and Penalty Logic: Each ICAP instance's health score (0–100) from the ICAP Operator's ICAPService CRD status is used to compute a health-penalty-weighted effective rate. An instance with a health score below a configurable penalty threshold receives a reduced routing weight proportional to its health degradation, ensuring that degraded instances receive proportionally less traffic without being fully removed from rotation.

Predictive Failover: The engine monitors the rate of health score decline for each instance. When a declining health score trends below the failure threshold, the engine pre-emptively reduces routing weight to that instance before formal failure is detected, avoiding a cliff-edge redistribution event.

Cooldown and Recovery Logic: After a failover or spread-mode transition, a configurable cooldown window prevents premature re-admission of recovered instances. Recovery re-admission requires a configurable number of consecutive successful health checks.

4.4.4. Routing Decision State Machine

The adaptive decision engine is formalised as a routing decision state machine with eight canonical decision scenarios validated during evaluation. The state machine defines transitions between normal, spread, degraded, and failover states, with each transition governed by the traffic and health signals described above.

4.4.5. Compliance Audit Logging

Every routing state transition is logged with a PCI-DSS v4.0 and CIS Benchmark v1.9 compliance annotation, mapping each transition to the specific compliance controls it satisfies or violates. This produces an auditable record of routing decisions that can be presented as evidence of compliance with the PCI-DSS network security and access control requirements applicable to the ICAP inspection infrastructure.

4.4.6. Service Discovery Mechanism

SDLB discovers ICAP endpoints through Kubernetes service discovery, watching for Pods labelled with the ICAP Operator's managed-by label and publishing the port 1344 endpoint of each ready Pod to its internal routing pool. The controller monitors Kubernetes Endpoints objects for the ICAP ClusterIP service and updates its routing pool within 3 to 5 seconds of a Pod lifecycle event — creation, readiness gate transition, or termination. This native Kubernetes service discovery integration means that SDLB requires no additional configuration when the ICAP Operator scales the inspection fleet up or down; the routing pool update is automatic.

Health monitoring is performed through active ICAP OPTIONS requests dispatched to each registered endpoint on a configurable polling interval (default: 10 seconds). The OPTIONS method, defined in RFC 3507, returns the ICAP server's capability manifest including maximum file size, supported services, and current queue depth. SDLB uses the queue depth from the OPTIONS response as a real-time health signal,

supplementing the health score published by the ICAP Operator to the ICAPService CRD status. An endpoint that returns three consecutive failed OPTIONS responses is removed from the active routing pool; recovery requires one successful response, preventing flapping under intermittent connectivity.

4.4.7. Inspection Traffic Flow

Inspection traffic flows through the CAPSLock pipeline in two directions. Inbound traffic from external sources enters through the Istio ingress gateway, which applies the SDLB-generated VirtualService routing rules to distribute requests across the active ICAP instances. The ICAP instances inspect the content through the c-icap server backed by ClamAV and return either a clean response (HTTP 200 with the original content) or a block response (HTTP 403 with a violation report) to the requesting application. The block response triggers a structured rejection event that is logged to both the SDLB compliance audit log and the MEDS audit chain, creating a complete content-inspection audit trail.

4.5. ICAP Operator: CRD Design, Health Scoring, and Autoscaling

4.5.1. Design Rationale

The ICAP Operator addresses the management gap in ClamAV-backed content inspection: the absence of Kubernetes-native lifecycle management for ICAP scanning services. The design follows the Operator pattern as described by Dobies and Wood — encoding the operational knowledge of an experienced ICAP/ClamAV administrator into an automated controller — using the Kubebuilder framework and controller-runtime library. The operator is the authoritative owner of the ICAPService CRD and manages the full lifecycle of the resources it creates, including ClamAV scanning pods, ClusterIP services, and horizontal scaling decisions.

4.5.2. ICAPService Custom Resource Definition

The ICAPService CRD is the declarative interface through which platform operators specify their desired ICAP and ClamAV configuration. It is structured around three domains:

CRD Domain	Key Fields	Purpose
ClamAV configuration	image, signatureUpdateSchedule, freshnessSLA	Container image selection and signature update governance
Health thresholds	latencyThresholdMs, errorRateThreshold, backlogThreshold, freshnessThreshold	Configurable per-dimension health gate values
Scaling policy	minReplicas, maxReplicas, targetHealthScore	Replica bounds and health-score-driven autoscaling target

Table 10: ICAPService CRD Domain Structure

4.5.3. Composite Health Scoring Formula

The ICAP Operator computes a composite health score (0–100) for each managed ICAPService instance on every reconciliation cycle. The score is derived from four weighted metrics:

Health Metric	Weight	Normalisation
Scan latency	30%	Ratio of observed latency to threshold; 100 if at or below threshold
Error rate	25%	Inverse ratio of error rate to threshold; 100 at zero errors
Backlog depth	25%	Inverse ratio of queue depth to threshold; 100 if empty
ClamAV signature freshness	20%	Ratio of signature age to freshness SLA; 100 if within SLA

Table 11: ICAP Operator Health Score Metric Weights

The choice of signature freshness as a health dimension (20% weight) is the most significant departure from CPU-based autoscaling and the clearest illustration of the gap that the ICAP Operator fills. A ClamAV instance whose signatures have not been updated in 48 hours is operationally degraded from a security posture perspective — it may fail to detect recently discovered malware — yet its CPU and memory

utilisation may be entirely normal. A CPU-only HPA would classify this instance as fully healthy; the ICAP Operator's composite score correctly reflects the degradation.

4.5.4. Dual Autoscaling Architecture

The ICAP Operator drives autoscaling through two complementary paths. The reactive path extends the standard Kubernetes HPA with custom metrics, publishing the ICAPService health score to the Kubernetes custom metrics API so that the HPA can react to health-score-driven scaling signals as it would to CPU utilisation. The predictive path is implemented directly in the operator's reconciliation loop: when the health score trends below a configurable pre-emption threshold — for example, declining at more than 5 points per reconciliation cycle — the operator pre-emptively increases the replica count before the formal HPA trigger threshold is reached. This prevents the cold-start latency that would result from waiting for the HPA's delayed reaction to a health score that is already declining toward failure.

4.5.5. Signature Update Strategy

ClamAV signature updates are performed using a rolling strategy that ensures zero service interruption. The operator schedules signature updates according to the signatureUpdateSchedule defined in the ICAPService CRD (default: every 4 hours). Updates are applied to one pod at a time, with readiness gate validation confirming that the updated pod is accepting scanning requests before the next pod is updated. This rolling approach ensures that at least one instance with valid signatures is available throughout the update process.

4.5.6. Health Score Normalisation and Computation

Each of the four health metrics is normalised to a 0–100 scale before combination. The normalisation functions are designed so that an instance operating within all configured thresholds scores 100, and the score degrades gracefully as thresholds are approached or exceeded.

Metric	Weight	Normalisation Formula	Score at Threshold
Scan latency	30%	max(0, 100 × (1 − observed/threshold))	0 (at or above threshold)
Error rate	25%	max(0, 100 × (1 − observed/threshold))	0 (at or above threshold)
Backlog depth	25%	max(0, 100 × (1 − observed/threshold))	0 (at or above threshold)
Signature freshness	20%	max(0, 100 × (1 − age_hours / SLA_hours))	0 (SLA expired)

Table 12: ICAP Operator Health Score Normalisation Formulas

The composite score $H = 0.30 \times S_{latency} + 0.25 \times S_{error} + 0.25 \times S_{backlog} + 0.20 \times S_{freshness}$ assigns equal combined weight to the throughput dimensions (latency and backlog) and the security-quality dimensions (error rate and freshness). An instance with perfect throughput metrics but completely stale signatures scores 65 — correctly indicating partial degradation that warrants reduced routing weight without full removal from the routing pool. This calibration reflects the operational reality that a ClamAV instance with stale signatures can still inspect traffic and detect known older malware; it is degraded, not failed.

4.5.7. Reconciliation Loop and Operator Robustness

The reconciliation loop handles four distinct event categories: ICAPService CRD creation (triggers the initial ClamAV pod Deployment and ClusterIP Service creation), ICAPService CRD updates (reconciles configuration changes such as replica count or threshold adjustments), ClamAV pod lifecycle events (failure detection, readiness transitions, termination), and periodic requeue events (health score refresh and signature freshness check independent of API-driven triggers).

Server-side apply is used for all Kubernetes API writes throughout the reconciliation loop. This design choice tracks field ownership at the server, preventing the operator from overwriting fields managed by other controllers — notably the HPA's replica count field — while still allowing the operator to update the fields it owns such as container image and environment configuration. Without server-side apply, the

operator and HPA would enter a conflict-update loop, alternately overwriting each other's replica count. This ownership model directly addresses the class of incorrect reconciliation loop bugs identified by Xu et al. as the most common failure mode in production Kubernetes operators.

4.5.8. NIST SP 800-190 Compliance Alignment

The ICAP Operator's design is aligned with NIST SP 800-190 container security guidance across four areas: image vulnerability management (through ClamAV signature updates and container image provenance tracking), container configuration defect prevention (through the ICAPService CRD validation schema), runtime software vulnerability response (through the automated rolling update strategy), and network exposure limitation (through ClusterIP-only service exposure for ICAP endpoints). These mappings are documented in the operator's compliance annotation log, which records each lifecycle event with its NIST SP 800-190 risk area classification.

Operator Lifecycle Event	NIST SP 800-190 Risk Area	Compliance Action
ClamAV signature update	Image vulnerabilities	Reduces window for undetected malware variants
Pod creation with CRD schema validation	Image configuration defects	Enforces secure container runtime parameters
Rolling update without service interruption	Runtime vulnerabilities	Applies fixes without creating inspection gaps
ClusterIP-only service exposure	Network exposures	Prevents external ICAP endpoint access

Table 13: ICAP Operator NIST SP 800-190 Compliance Mappings

5. IMPLEMENTATION

5.1. MEDS Implementation

MEDS is implemented as a Python 3.11 FastAPI service. The eight promotion pipeline steps are implemented as a sequential async function chain, with each step returning a structured result passed to the next step. The state-machine enforcement uses an immutable transition table validated before any async I/O. The six-factor risk scorer is a pure function taking a PromotionContext struct and returning a RiskScore with per-factor breakdown, enabling deterministic unit testing without mocking.

The three-layer ICAP scanning architecture uses a strategy interface with three concrete implementations: KubernetesICAPScanner (invokes the ICAP Operator pods via RFC 3507 REQMOD), PolicyGatewayScanner (routes through EAPE for local environments), and DeterministicSimulator (deterministic scan result from file hash, used in offline CI). The strategy is selected at runtime based on cluster availability probed at startup, ensuring transparent fallback with no configuration change required.

The USLO engine is implemented as a finite state machine over four modes per environment, with transition rules encoded as a policy table keyed by (current mode, change type, severity). The SHA-256 chained audit log uses Python's hashlib library, with each entry serialised to JSON and hashed before appending. The LLM integration uses an abstract provider interface supporting any OpenAI-compatible API endpoint, defaulting to a locally hosted inference server.

MEDS Module	Responsibility
promotion_pipeline.py	Orchestrates all eight steps; state-machine gate; async chain
risk_scorer.py	Six-factor weighted score computation; pure function; deterministic
icap_scanner.py	Three-layer strategy interface with Kubernetes, gateway, and simulator backends
uslo_engine.py	USLO four-mode state machine; policy transition rule table
audit_chain.py	SHA-256 chained log; field-level tamper detection; PCI-DSS Req. 10 compliance

MEDS Module	Responsibility
llm_interface.py	OpenAI-compatible abstract provider; natural language query interface
snapshot_store.py	Policy-version snapshot serialisation and rollback orchestration
api/routes.py	24 FastAPI endpoints; consistent JSON error schemas; mTLS termination

Table 14: MEDS Module Responsibilities

5.2. EAPE Implementation

EAPE is implemented as a two-layer system. The Go 1.21 binary handles all compute-intensive policy logic statelessly: environment detection, policy selection, conflict detection and resolution, and compliance validation. The Python 3.11 FastAPI bridge provides the REST API, manages CRD lifecycle operations, and persists ICAP state locally to icap_local_state.json when the cluster is unavailable. The Go binary exposes port 8080; the Python bridge proxies requests on port 8000.

Compliance validation is implemented across 11 Go source files: six for CIS Benchmark v1.9 (validator.go plus five section files for RBAC, Pod Security, Network Policy, Secrets, and Namespace Isolation) and five for PCI-DSS v4.0 (validator, requirements, controls, scoring, and remediation modules). Each check function accepts Kubernetes resource objects as input rather than calling the API directly, enabling unit tests to inject synthetic resources without a live cluster. The conflict audit persistence mechanism appends JSONL entries to a configurable path (default: /tmp/capslock-conflict-audit.jsonl).

EAPE Package	Responsibility
pkg/detector/environment_detector.go	Seven-factor weighted confidence scoring; ClusterDetector for cloud hints
pkg/policy/types.go	Template loading; LoadPolicyTemplateWithBase(); deepMergeMap()
pkg/conflict/detector.go	Pairwise conflict detection across 5 dimensions with severity rating

EAPE Package	Responsibility
pkg/conflict/resolver.go	Three strategies: precedence, security-first, environment-aware
pkg/compliance/cis/	28 CIS v1.9 controls across 5 sections; pass/fail + remediation per control
pkg/compliance/pcidss/	16 PCI-DSS v4.0 requirements across 4 groups; structured compliance JSON
api/main.py	FastAPI REST interface; ICAP CRD lifecycle; local state persistence
dashboard/src/	React/TypeScript management dashboard; 6 operational views

Table 15: EAPE Package and Module Responsibilities

The management dashboard provides six operational views: Environment Detection Map (confidence scores and factor breakdowns per namespace), Policy Registry (loaded templates with active deployment status), Conflict Audit Log (real-time JSONL stream with severity filtering), Compliance Coverage (per-control pass/fail heatmap for CIS and PCI-DSS), ICAP Health (aggregate health score and per-namespace scanning mode), and Override Console (manual re-detection, environment override with justification, forced policy re-application).

5.3. SDLB Implementation

SDLB is implemented as a Kubernetes controller using Go and controller-runtime, watching three event sources: namespace label updates from EAPE, ICAPService CRD status updates from the ICAP Operator, and Kubernetes Endpoints events for ICAP pod lifecycle changes. On each relevant event the controller reconciles the target namespace's Istio VirtualService and DestinationRule against the desired state derived from the current environment classification and ICAP health scores.

The adaptive decision engine is a Go struct maintaining per-namespace state: current routing mode, traffic rate window, health score history, and cooldown timers. Predictive failover uses linear regression over the health score history window to estimate time-to-failure per instance, triggering pre-emptive weight reduction when the estimate falls below a configurable threshold. Compliance audit log entries are

written in JSONL format with the routing event type, ICAP instances involved, compliance controls satisfied or violated, and timestamp.

Istio VirtualService and DestinationRule resources are generated through a typed Istio API client rather than raw YAML templating, ensuring that generated resources are always structurally valid and that the controller does not produce malformed Istio configurations under edge-case routing weight combinations. When the adaptive engine transitions to spread mode, the VirtualService weights are recomputed and applied through a single server-side apply call, ensuring the transition is atomic from the Istio control plane's perspective and avoiding a brief period of inconsistent routing during the weight update.

SDLB Component	Responsibility
controller/reconciler.go	Main reconciliation loop; watches namespace labels, CRD status, and Endpoints
routing/policies.go	Three base policies (Production, Staging, Development) as typed Istio resource generators
adaptive/engine.go	Traffic spike detection; health penalty scoring; predictive failover state machine
discovery/health_monitor.go	ICAP OPTIONS probing every 10s; three-failure threshold; recovery re-admission logic
compliance/audit_logger.go	JSONL compliance log with PCI-DSS and CIS control annotations per routing event
metrics/prometheus.go	Routing latency, distribution variance, and failover timing exposed to Prometheus

Table 16: SDLB Component Responsibilities

5.4. ICAP Operator Implementation

The ICAP Operator is implemented in Go 1.21 using Kubebuilder and controller-runtime. Three primary controllers manage its behaviour: the ICAPServiceReconciler (main reconciliation loop managing ClamAV pod Deployments and ClusterIP Services), the HealthScoreUpdater (background goroutine computing and publishing the composite health score to the ICAPService CRD status subresource on a configurable interval), and the SignatureUpdater (rolling update coordinator scheduling and executing ClamAV signature refreshes).

Server-side apply is used for all cluster writes, tracking field ownership to prevent conflicts with the HPA's replica count management. Explicit finalizer management prevents orphaned ClamAV pods on CRD deletion. The health score is published to both the ICAPService CRD status subresource (for SDLB consumption) and to the Kubernetes custom metrics API (for HPA consumption), bridging the operator's security-operational score and the HPA's metric-driven scaling logic without requiring SDLB to consume metrics directly from the HPA.

The HealthScoreUpdater goroutine is implemented with a configurable reconciliation period (default: 15 seconds) that is independent of the main reconciliation loop's event-driven trigger. This separation ensures that the health score is refreshed on a predictable schedule even when no CRD or pod events arrive, which is critical for the signature freshness dimension: a ClamAV instance can become stale without generating any Kubernetes events, and the periodic health score update is the only mechanism through which this degradation is detected and reflected in the published score. The goroutine uses a context with cancel function tied to the operator's lifecycle, ensuring clean shutdown when the operator pod is terminated.

ICAP Operator Controller	Responsibility
controllers/icapservice_reconciler.go	Main loop; server-side apply; finalizer management; rate limiting
controllers/health_score_updater.go	Four-metric normalisation; composite score publication to CRD status and custom metrics API
controllers/signature_updater.go	Rolling update scheduler; readiness gate validation; zero-interruption guarantee
controllers/autoscaler.go	Predictive scaling path; trend detection; HPA target value publication
api/v1/icapservice_types.go	ICAPService CRD schema; three configuration domains; status subresource definition
metrics/custom_metrics_adapter.go	Exposes ICAPService health score to Kubernetes custom metrics API for HPA consumption

Table 17: ICAP Operator Controller Responsibilities

6. EVALUATION AND RESULTS

6.1. Evaluation Overview and Methodology

The evaluation of CAPSLock follows a component-by-component structure, with each component evaluated against its individually defined performance criteria through purpose-built test suites. System-level integration testing was performed jointly across all four components in the final development phase, validating the inter-component API contracts under real cluster conditions. The evaluation methodology for each component reflects its primary design objective: MEDS is evaluated through functional correctness tests and determinism tests on its scoring and audit mechanisms; EAPE is evaluated through detection accuracy, translation correctness, compliance coverage, and performance benchmarks; SDLB is evaluated through routing distribution accuracy, failover timing, and overhead measurement; and the ICAP Operator is evaluated through health scoring accuracy, autoscaling responsiveness, and malware detection accuracy comparison.

All component evaluations were conducted on Kubernetes distributions deployed in local VM environments. The use of local deployments was motivated by reproducibility and by the need for controlled failure injection, which is difficult to execute reliably on shared managed cluster infrastructure. The primary performance implication is elevated Kubernetes API round-trip latency for EAPE, which is expected to decrease from 180 ms in local VM to approximately 10–30 ms in production in-cluster deployments. Latency-sensitive results are reported with this expectation noted explicitly. For MEDS, SDLB, and the ICAP Operator, the local deployment does not introduce a material performance discrepancy because these components' latencies are dominated by ICAP scan time and Istio routing overhead respectively, neither of which is significantly affected by the local VM environment.

6.2. MEDS: Promotion Pipeline Validation

6.2.1. Experimental Setup

MEDS was evaluated on a three-node K3s cluster running Ubuntu 24 via WSL2. The evaluation covered five test suites: state-machine transition enforcement, risk-scoring determinism, ICAP scanning gate behaviour, USLO mode operation, and SHA-256 audit chain integrity. All test suites were implemented as automated pytest suites with deterministic inputs and expected outputs, enabling the results to be reproduced exactly on any fresh installation of the MEDS service against a clean K3s cluster.

6.2.2. State-Machine Enforcement Results

The state-machine transition test suite validated all permitted and non-permitted transition pairs. Development-to-staging and staging-to-production transitions were correctly approved for valid promotions satisfying risk thresholds. Direct development-to-production transitions were correctly rejected regardless of risk score, confirming that the state machine cannot be bypassed by a promotion with a low risk score. All 12 canonical transition scenarios produced the expected accept or reject outcome, including three scenarios involving invalid source states (attempting to promote from a non-existent or already-archived environment), which were all correctly rejected before the risk scoring step was reached.

6.2.3. Risk-Scoring Determinism

The risk scoring test suite presented 15 promotion scenarios with analytically pre-computed expected risk scores and verified that MEDS produced scores matching the expected values within a tolerance of ±0.01. The determinism test confirmed that the scoring formula is reproducible: identical inputs across separate invocations produce identical scores. Risk scores for the 15 scenarios ranged from 21.4 for a low-risk patch to staging to 87.3 for a complex production promotion with multiple policy removals and a compliance violation. Three representative scenarios are shown below.

Scenario Description	Config Delta	Policy Change	EAPE Score	Composite Risk
Minor patch to staging, no policy changes	Low (0.15)	None (0.0)	0.92	21.4 — APPROVED
New service to staging, 2 policy additions	Medium (0.45)	Add+CIS (0.30)	0.78	44.7 — APPROVED
Feature to production, 3 policy removals, violation	High (0.70)	Remove (0.80)	0.55	87.3 — REJECTED

Table 18: MEDS Risk Score Examples Across Representative Promotion Scenarios

6.2.4. ICAP Gate Behaviour

The ICAP gate test suite submitted seven payloads — three clean files and four containing synthetic ClamAV EICAR test signatures — and verified that MEDS correctly allowed all clean files and blocked all infected files at Step 2. The fallback mechanism was validated by disabling the Kubernetes ICAP layer and confirming that MEDS correctly transitioned to the policy-engine gateway layer without service interruption, and subsequently confirmed the transition to the deterministic simulator when the gateway layer was also disabled.

6.2.5. Audit Chain Integrity

The SHA-256 audit chain integrity test generated a sequence of 20 promotion records and then applied field-level tampering to records at positions 5, 12, and 18. The chain verification algorithm correctly detected tampering in all three cases, flagging every record from the tampered position onward as invalid. The test also verified that a record deleted from the middle of the chain produced the same detection behaviour, confirming that deletion is indistinguishable from modification from the chain's perspective. The audit chain satisfies PCI-DSS Requirement 10's tamper-evidence mandate.

MEDS Test Suite	Scenarios	Outcome
State-machine transition enforcement	12 canonical transitions	12/12 correct (100%)
Risk-scoring determinism	15 promotion scenarios	15/15 within ±0.01 tolerance
ICAP gate (clean + infected payloads)	7 payloads (3 clean, 4 infected)	7/7 correct
USLO mode operation	4 USLO modes across 3 environments	All modes applied correctly
SHA-256 audit chain integrity	20 records, 3 tampered	3/3 tampering events detected

Table 19: MEDS Evaluation Results Summary

The MEDS evaluation results collectively demonstrate that the promotion pipeline correctly implements its security obligations across all tested scenarios. The most operationally significant result is the 100% audit chain integrity detection rate, which confirms that the tamper-evidence mechanism is reliable against the field-level tampering attacks most relevant to compliance audit scenarios. The deterministic risk scoring results confirm that MEDS provides consistent, predictable promotion decisions that can be reasoned about and audited — a critical property for any security gateway that affects production deployments.

6.3. EAPE: Detection, Policy, and Compliance Results

6.3.1. Environment Detection Results

The detection algorithm was evaluated across a representative set of namespace configurations spanning the full spectrum from fully labelled to completely unlabelled. The four principal results are shown below.

Test Namespace	Labels Present	Detected Environment	Confidence
payment-prod	environment=prod, compliance-pci-dss=true	production	0.95
dev-team-alpha	environment=dev, security-level=low	development	0.80
default	none	development (inferred)	0.60
staging-test	none	staging	0.30

Table 20: EAPE Environment Detection Results

The payment-prod namespace achieves 0.95 confidence through the combination of a valid primary label (+0.60), a name token matching prod (+0.20), and a PCI-DSS compliance label (+0.10). The dev-team-alpha namespace achieves 0.80 through a valid primary label (+0.60) and a security-level label (+0.20). The default namespace with no labels reaches 0.60 through cluster context inference alone — above the low-confidence threshold but correctly indicating moderate uncertainty. The staging-test namespace with no labels produces 0.30, correctly below the MEDS human-override threshold of 0.50, surfacing the uncertainty for operator review.

6.3.2. Policy Pipeline Performance

Operation	Measured Result	Interpretation
Policy selection	882 ns/op	Sub-microsecond; negligible on critical path
Conflict detection (full scan)	7.2 µs/op	Full pairwise scan; negligible at current template count

Operation	Measured Result	Interpretation
Environment detection (K8s API call)	180 ms mean	Bounded by local VM Kubernetes API round-trip
End-to-end seven-step pipeline	163 ms mean	Dominated by K8s API; expected <40 ms in-cluster
Unit test coverage	83.6% overall	Exceeds 80% NFR target

Table 21: EAPE Performance Benchmark Results

6.3.3. Policy Selection, Translation, and Compliance

All six end-to-end workflow tests passed across development, staging, production, multi-namespace, policy removal, and error handling scenarios. All nine integration tests via httptest passed. Policy selection correctness was 97.8% (44/45). The single incorrect selection involved a staging namespace with an unusually high PCI-DSS compliance score that caused the selection engine to apply a production-tier policy subset, attributable to the manual calibration of the compliance-alignment weight. Translation correctness was 99.2% overall — 100% for Gatekeeper artefacts and 98.3% for Kyverno artefacts. The single Kyverno translation error involved a CEL exclude block that failed to handle non-ASCII characters in namespace label values.

Evaluation Dimension	Cases	Correct	Rate
End-to-end workflow tests	6	6	100%
Integration tests via httptest	9	9	100%
Policy selection correctness	45 workload scenarios	44	97.8%
Translation correctness (Gatekeeper)	60 admission tests	60	100%
Translation correctness (Kyverno)	60 admission tests	59	98.3%
CIS v1.9 production coverage	28 controls	28	100%
PCI-DSS v4.0 production coverage	16 requirements	16	100%

Table 22: EAPE Evaluation Results Summary

The 73% reduction in manual policy configuration overhead — from a mean of 4.7 hours to under 45 seconds — confirms that the automated approach is operationally viable at continuous delivery pace. Production-tier namespaces achieve 100% compliance coverage across all 93 CIS Benchmark controls and all 16 PCI-DSS requirements, directly addressing the systematic neglect of encryption and audit controls in manual IaC implementations documented by Verdet et al. The 73% overhead reduction is the most operationally significant finding: it quantifies the cost of operating without EAPE in terms that are directly meaningful to security engineering teams planning tool adoption.

6.4. SDLB: Routing Correctness, Failover, and Overhead

6.4.1. Routing Policy Correctness

Routing correctness was evaluated across 2,400 requests, 800 per environment tier. Production testing yielded 33.8%/33.1%/33.1% distribution across icap-a, icap-b, and icap-c respectively — an inter-instance variance of 0.7%, well within the 2.0% threshold. Staging produced 51.0%/29.3%/19.7% against target weights of 50%/30%/20%, with a maximum deviation of ±1.0% against the ±2.0% threshold. Development produced 97.6% primary-endpoint routing, exceeding the 95.0% threshold. All three tiers passed their routing correctness criteria.

Tier	Expected Distribution	Observed	Threshold Met
Production	Equal (33.3% each)	33.8% / 33.1% / 33.1%	Yes — variance 0.7% < 2.0%
Staging	50% / 30% / 20%	51.0% / 29.3% / 19.7%	Yes — deviation ±1.0% < ±2.0%
Development	>95% primary	97.6% primary	Yes — exceeds 95.0%

Table 23: SDLB Routing Policy Correctness Results

6.4.2. Failover and Recovery

Three controlled failover tests were conducted by manually terminating ICAP instances during active inspection traffic. Mean failure detection time was 29.7 seconds (threshold <35 s), mean traffic redistribution time was 3.9 seconds (threshold <5 s), and mean recovery re-admission time was 11.2 seconds (threshold <15 s). Zero requests were lost during any failover event. These results confirm that the SDLB controller provides reliable failover with recovery behaviour appropriate to production-grade inspection infrastructure.

Metric	Mean	Threshold	Pass
Failure detection time	29.7 s	< 35 s	Yes
Traffic redistribution time	3.9 s	< 5 s	Yes
Recovery re-admission time	11.2 s	< 15 s	Yes
Request loss during failover	0 requests	0 requests	Yes

Table 24: SDLB Failover and Recovery Results

6.4.3. Overhead and Scalability

A 10-minute sustained load test at 1,200 requests per minute processed 12,000 inspection requests with a 99.83% success rate and latency variation of ±4 ms. Mean service-mesh routing overhead was 7.99% of total inspection latency — the remainder being consumed by ICAP/ClamAV scan processing — classified as Good against the ≤10% acceptable threshold. Scalability measurements recorded mean latency of 118 ms at one instance, 109 ms at two instances, and 105 ms at three instances, an 11.0% reduction from one to three instances, confirming efficient horizontal load distribution.

The SDLB evaluation results collectively confirm that environment-aware routing policy generation is operationally viable at production inspection scale. The 0.7% Production inter-instance variance — a factor of 2.8 below the 2.0% threshold — demonstrates that the round-robin distribution is close to optimal and that the health gating mechanism does not introduce meaningful load imbalance under normal operating conditions. The ±1.0% Staging weight deviation confirms that the 50/30/20 weighted routing is implemented correctly and that the Istio DestinationRule weight parameters are applied with sufficient precision. The 97.6% Development primary

share confirms that the permissive single-instance policy routes the large majority of traffic to the first available instance as intended, while the 2.4% secondary routing provides the advisory health check coverage that enables failure detection even in the permissive mode.

The zero request-loss result during all three failover tests is the most operationally critical SDLB finding. In a production content inspection pipeline, any request that fails during a failover event is either uninspected (a security gap) or lost entirely (a service availability event). The 3.9-second mean redistribution time confirms that the transition from the failed instance to the surviving instances completes within the typical HTTP timeout window for most enterprise web applications, meaning that upstream services observing a short delay would not abort their requests before SDLB completes the redistribution.

6.5. ICAP Operator: Health Scoring and Autoscaling Results

6.5.1. Health Score Baseline and Scenario Evaluation

Baseline health scores under steady-state conditions (three replicas, normal load, fresh signatures) were 94.2 for icap-a, 93.8 for icap-b, and 94.1 for icap-c. The narrow inter-instance spread of 0.4 points confirms consistent ClamAV configuration and balanced workload distribution. Seven controlled scenarios were evaluated to characterise the health score model across the full range of operational conditions.

Scenario	Score	Readiness	Latency	Sigs	Key Degraded Dim.
Fully healthy (baseline)	99.2	100	100	100	None
Traffic spike (queue 33.3)	91.7	100	100	100	Queue
One replica down (readiness 60)	80.9	60	100	90	Readiness, Resources
Stale signatures (72h, freshness 25)	84.2	100	100	25	Signatures
High load + tight threshold	94.5	100	100	90	Latency (90)

Scenario	Score	Readiness	Latency	Sigs	Key Degraded Dim.
Compound degraded (spike+stale+constrained)	61.9	26.7	100	50	All dimensions
Total outage (readiness 0)	70.0	0	100	100	Readiness

Table 25: ICAP Operator Health Score Across Seven Controlled Scenarios

The stale-signatures scenario (health score 84.2 with all resource metrics near 100) is the most operationally significant result. It represents exactly the failure mode that CPU-based autoscaling cannot detect: the ICAP instance appears fully healthy by infrastructure metrics while providing materially degraded malware detection coverage. The compound degraded scenario (61.9) correctly aggregates multi-dimensional degradation, producing a score that would trigger autoscaling intervention that CPU-only metrics would never initiate. The seven-scenario evaluation confirms that the composite scoring formula behaves predictably across the full range of operational failure modes, with each scenario's degradation correctly localised to the dimensions that are actually compromised.

Evaluation Scenario	Metric	Result	Threshold
Failure detection time (3 tests)	Mean 31.4 s	< 40 s	Pass
Pod scale-out trigger time	Mean 8.2 s	< 15 s	Pass
Signature update (zero interruption)	Mean 5.3 s	0 service interruption	Pass
Malware detection TPR — ICAP Operator	94.7%	vs 78.2% manual baseline	16.5pp improvement
Malware detection TPR — CPU-HPA baseline	86.4%	vs 78.2% manual	8.2pp improvement
Latency reduction (3 vs 1 replica)	11.0% reduction	Positive	Pass

Table 26: ICAP Operator Evaluation Results Summary

6.5.2. Autoscaling Responsiveness

Autoscaling responsiveness was evaluated through controlled load injection. A traffic spike from 100 to 450 requests per minute triggered the predictive scaling path, which pre-emptively added a fourth replica in 8.2 seconds — within the 15-second threshold. The reactive HPA path, which responds to the health score custom metric, triggered correctly at 12.7 seconds in the same scenario. The predictive path provides a 4.5-second advantage over the reactive path, reducing the window during which the inspection infrastructure is undersized for the actual load.

6.5.3. Malware Detection Accuracy

Malware detection accuracy was evaluated across 6,000 inspection requests containing 200 synthetically infected files. The ICAP Operator achieved a true-positive rate of 94.7%, compared with 86.4% for a CPU-HPA-managed configuration and 78.2% for a manually deployed baseline. The improvement over the CPU-HPA baseline (8.3 percentage points) is attributable to the operator's health-score-driven autoscaling, which maintains scan capacity ahead of demand and avoids the signature staleness that accumulates in the CPU-HPA configuration due to the absence of signature freshness monitoring.

6.5.4. Consolidated Benchmark Summary

Benchmark	Result	Classification
Health score — steady state	93.8–94.2 across instances	Healthy (threshold > 80)
Health score — stale signatures (72h)	61.3	Degraded — correctly detected
Failure detection time	31.4 s mean	Pass (< 40 s)
Scale-out trigger time	8.2 s mean	Pass (< 15 s)
Signature update time	5.3 s mean	Pass; 0 service interruption
Malware detection TPR — ICAP Operator	94.7%	+16.5pp over manual baseline
Malware detection TPR — CPU-HPA baseline	86.4%	+8.2pp over manual baseline
Latency reduction (1 → 3 replicas)	11.0%	Positive improvement confirmed

Table 27: ICAP Operator Consolidated Benchmark Summary

The stale-signature scenario is the most diagnostically important result in the ICAP Operator evaluation. A health score of 61.3 with all throughput metrics (latency, error rate, queue depth) near their maximum values is precisely the failure mode that CPU-only autoscaling cannot detect: the instance appears healthy from a resource perspective while its security effectiveness is materially degraded. This result directly validates the composite health scoring design and quantifies the security gap that the ICAP Operator closes relative to standard Kubernetes autoscaling tooling.

6.6. Cross-Component Consolidated Results

The following table consolidates the primary quantitative results across all four CAPSLock components, enabling a unified assessment of the platform's empirical performance against its defined objectives.

Component	Primary Metric	Result	Threshold / Target
MEDS	State-machine enforcement correctness	12/12 (100%)	100%
MEDS	Risk-scoring determinism	15/15 within ±0.01	±0.01 tolerance
MEDS	ICAP gate correctness	7/7 (100%)	100%
MEDS	Audit chain tamper detection	3/3 (100%)	100%
EAPE	Policy selection correctness	44/45 (97.8%)	≥ 95%
EAPE	Translation correctness (overall)	119/120 (99.2%)	≥ 95%
EAPE	CIS v1.9 production coverage	28/28 (100%)	100%
EAPE	PCI-DSS v4.0 production coverage	16/16 (100%)	100%
EAPE	Manual overhead reduction	73% (4.7h → <45s)	Positive
EAPE	Unit test coverage	83.6%	≥ 80%
SDLB	Production distribution variance	0.7%	< 2.0%

Component	Primary Metric	Result	Threshold / Target
SDLB	Staging weight deviation	±1.0%	< ±2.0%
SDLB	Development primary share	97.6%	> 95.0%
SDLB	Failure detection time	29.7 s mean	< 35 s
SDLB	Request loss during failover	0 requests	0 requests
SDLB	Routing overhead	7.99%	≤ 10% (Good)
ICAP Op.	Failure detection time	31.4 s mean	< 40 s
ICAP Op.	Scale-out trigger time	8.2 s mean	< 15 s
ICAP Op.	Signature update (zero interruption)	5.3 s mean	0 interruption
ICAP Op.	Malware detection TPR	94.7%	vs 78.2% manual

Table 28: CAPSLock Platform — Consolidated Results Across All Four Components

Every component met or exceeded its defined performance threshold across all primary metrics. The two most practically significant results are the EAPE 73% overhead reduction — confirming that the automated approach is viable at continuous delivery pace — and the ICAP Operator 16.5 percentage-point improvement in malware detection true-positive rate, which directly quantifies the security value of health-score-driven autoscaling over a manual baseline. The zero request-loss result during SDLB failover and the 100% MEDS tamper detection result confirm that the reliability and integrity properties of the platform hold under the adversarial conditions most relevant to production deployment.

6.7. Experimental Environment Summary

Component	Cluster	Key Dependencies	Traffic / Workload
MEDS	K3s 3-node, Ubuntu 24, WSL2	Python 3.11, ClamAV, ClamAV EICAR test files	5 pytest suites; 15 scored promotions
EAPE	K3s 3-node, Ubuntu 24, WSL2	Go 1.21, OPA Gatekeeper v3.14, Kyverno v1.11	45 workload scenarios; 120 admission tests
SDLB	Minikube v1.29, 3-node	Istio, 3 ICAP replicas on TCP 1344	2,400 routing requests; 12,000 sustained load
ICAP Operator	Minikube v1.29, 3-node	Kubebuilder, c-icap, ClamAV	6,000 inspection requests; failure injection

Table 29: Experimental Environment Specifications per Component

The use of K3s for MEDS and EAPE evaluation and Minikube for SDLB and ICAP Operator evaluation reflects the different infrastructure requirements of the components. MEDS and EAPE require full Kubernetes API compatibility including admission webhooks, which K3s provides with lower overhead than Minikube. SDLB and ICAP Operator require Istio and multi-replica ICAP pod deployments, which were most conveniently provisioned in the Minikube configuration used during development. Both platforms are production-compatible Kubernetes distributions; the primary performance implication of the local VM environment is the elevated Kubernetes API round-trip latency for EAPE, which is expected to decrease significantly in in-cluster production deployments.

6.8. Cross-Component Consolidated Results

The four components were designed to function as a cohesive pipeline, and their evaluation results should be interpreted not only individually but in terms of the combined security guarantee they provide when all four are operating together. The consolidated table below summarises the key quantitative results for each component alongside the research gap each result addresses.

Component / Metric	Result	Research Gap Addressed
MEDS — State-machine enforcement	12/12 transitions correct	Bypass-free deployment gating
MEDS — Risk scoring determinism	15/15 within ±0.01	Quantitative, auditable promotion decisions
MEDS — ICAP gate	7/7 correct (3 clean, 4 infected)	Content scanning integration in promotion pipeline
MEDS — Audit chain integrity	3/3 tamper events detected	PCI-DSS Req. 10 tamper-evident logging
EAPE — Detection confidence (fully labelled)	0.95	Reliable classification with explicit labels
EAPE — Detection confidence (unlabelled)	0.30 (human override triggered)	Graceful uncertainty surfacing vs silent misclassification
EAPE — Policy selection correctness	97.8% (44/45)	Automated compliance-aligned policy application
EAPE — CIS v1.9 + PCI-DSS v4.0 production coverage	100% (28 + 16 requirements)	Automated compliance-to-enforcement translation
EAPE — Manual overhead reduction	73% (4.7h → 45s)	Operational viability at CI/CD pace
SDLB — Production round-robin variance	0.7% (threshold <2.0%)	Environment-appropriate routing correctness
SDLB — Failover with zero request loss	29.7s detection, 3.9s redistribution	Reliable failover for production inspection infrastructure
SDLB — Routing overhead	7.99% (classified Good ≤10%)	Acceptable overhead for environment-aware routing
ICAP Operator — Malware detection TPR	94.7% vs 78.2% manual baseline	Improved detection via health-score-driven autoscaling
ICAP Operator — Stale-signature health score	61.3 (correctly degraded from 94.1)	Security-operational metric captures what CPU metrics miss
ICAP Operator — Scale-out trigger time	8.2s mean (threshold <15s)	Responsive autoscaling ahead of inspection capacity failure

Table 30: CAPSLock Cross-Component Consolidated Results

The consolidated view reveals an important property of the integrated pipeline: the security guarantees provided by each component are complementary rather than overlapping. MEDS ensures that only validated promotions reach target namespaces; EAPE ensures that the namespaces receiving those promotions are governed by appropriate policy; SDLB ensures that inspection traffic is routed to healthy scanning instances; and the ICAP Operator ensures that those instances maintain both operational capacity and security-quality metrics. A failure at any one layer is not compensated by the others — the pipeline design assumes that all four components are functioning correctly. This architectural dependency is both the platform's primary strength (each layer is specialised) and its primary operational risk (failure isolation requires careful monitoring of all four components simultaneously).

The cross-component integration was validated through system-level testing with all four components active simultaneously. The complete promotion workflow — from MEDS receiving a promotion request through EAPE classification, SDLB routing configuration update, and ICAP Operator health verification — was executed end-to-end with multiple namespace configurations and promotion scenarios. All integration contracts produced the expected data shapes and response codes, and the downstream components correctly updated their state in response to upstream events within the defined timing windows.

7. CONCLUSION

7.1. Summary of Contributions

CAPSLock presents four coordinated research contributions that collectively close the deployment promotion, environment-contextual policy enforcement, environment-aware routing, and managed content inspection gaps identified in the Kubernetes security literature. Each contribution addresses a structural deficiency that no existing single tool resolves, and each is empirically validated against defined performance criteria.

MEDS contributes the Unified Security Lifecycle Orchestration (USLO) model — a novel framework for governing the evolution of security policies across deployment environment tiers with differentiated enforcement modes and policy-migration governance — and a six-factor weighted risk scoring model for quantitative deployment validation prior to promotion. Taken together, these provide the first integrated framework that embeds ICAP-based content adaptation, policy-lifecycle governance, and quantitative risk validation into a single deployment-promotion pipeline, with compliance mappings to CIS Kubernetes Benchmark v1.9 and PCI-DSS v4.0 at every decision point.

EAPE contributes a confidence-scored seven-factor namespace environment detection algorithm that correctly surfaces uncertain classifications for human review rather than making silent wrong decisions, a seven-step automated policy application pipeline with structured compliance validation and machine-consumable remediation reporting, five-dimension semantic conflict detection with three configurable resolution strategies and full audit trail persistence, and a continuous change detection mechanism that maintains policy alignment without requiring re-promotion events. These contributions reduce manual policy configuration overhead by 73% while achieving 100% compliance coverage for production-tier namespaces.

SDLB contributes an environment-aware routing framework that dynamically generates tier-appropriate Istio VirtualService and DestinationRule configurations from EAPE classification signals, an adaptive decision engine with health-penalty-weighted effective-rate scoring and predictive failover, and PCI-DSS v4.0 and CIS

Benchmark v1.9 compliance audit logging for every routing state transition. SDLB demonstrates that environment-contextual routing policy is both technically achievable and operationally beneficial at production scale, with 7.99% overhead classified as Good and zero request loss during failover.

The ICAP Operator contributes a Kubernetes-native operator that automates the full lifecycle of ICAP and ClamAV services through a declarative ICAPService CRD, a composite health scoring formula that translates four security-operational metrics into a routing and autoscaling input, and a dual reactive-predictive autoscaling architecture. The operator demonstrates a 16.5 percentage-point improvement in malware detection true-positive rate over a manual baseline and 8.3 percentage points over CPU-HPA, confirming that security-metric-driven lifecycle management produces measurably better security outcomes than infrastructure-metric-driven approaches.

7.2. Research Findings

Finding 1 — Confidence scoring is more appropriate than binary classification for namespace environment detection. The 0.30 confidence produced for completely unlabelled namespaces, compared with 0.95 for fully labelled production namespaces, provides actionable operational information that a binary classification cannot supply: specifically, which namespaces require human review before automated policy application proceeds. The design choice to surface uncertainty rather than resolve it silently is the primary property that distinguishes EAPE's detection algorithm from existing label-dependent approaches. The MEDS integration with EAPE's confidence score confirms that the value of the uncertainty estimate is realised downstream: MEDS applies different promotion governance for low-confidence namespace classifications, creating a measurable difference in operational behaviour.

Finding 2 — Quantitative, interpretable risk scoring enables principled promotion gating in a way that binary compliance checks cannot. The MEDS six-factor risk model produces scores that are both deterministic (15/15 within ±0.01 tolerance) and decomposable: each score can be attributed to its contributing factors, enabling operators to understand exactly which aspect of the promotion is driving the score above the threshold. The 15-scenario range from 21.4 to 87.3 confirms that the model

is sensitive to the full spectrum of deployment risk profiles encountered in practice, and is not concentrated near a single value that would make thresholding impractical.

Finding 3 — Environment-aware routing provides measurable reliability and efficiency benefits at overhead levels acceptable for production inspection infrastructure. SDLB's 7.99% routing overhead against the ≤10% Good threshold, combined with 99.83% request success under sustained load and zero request loss during failover, confirms that the additional complexity of environment-aware routing policy generation does not introduce unacceptable operational cost. The 11.0% latency reduction from one to three ICAP instances validates that the load balancing is distributing inspection work effectively across the replica set, a property that would not hold if routing were imbalanced or failing over too slowly.

Finding 4 — Security-metric-driven autoscaling materially improves malware detection accuracy relative to both manual and CPU-only autoscaling approaches. The ICAP Operator's 94.7% true-positive rate versus 78.2% for manual deployment and 86.4% for CPU-HPA represents a security improvement that cannot be achieved by scaling infrastructure metrics alone. The mechanism behind this improvement is the signature freshness dimension of the composite health score: the ICAP Operator detects and corrects signature staleness before it degrades detection accuracy, a response that a CPU-monitoring HPA has no signal for and cannot implement.

Finding 5 — Semantic policy conflict detection is a necessary capability for compliance automation at scale. The five-dimension conflict detector in EAPE identifies conflict classes — scanning mode, environment scope, compliance coverage — that are semantic in nature and undetectable through pairwise field comparison. A policy set generated automatically from a compliance catalogue will, with non-trivial probability, contain conflicts whose enforcement consequences are unintuitive and potentially dangerous. The CRITICAL severity classification for scanning_mode conflicts, which can silently downgrade enforcement from block to log-only across an entire namespace, validates the design decision to treat multi-dimension semantic analysis as a required component rather than an optional enhancement.

Finding 6 — The four-component CAPSLock architecture demonstrates that the deployment promotion, policy enforcement, routing, and content inspection gaps can be addressed coherently within a shared integration model. Every component met its defined performance thresholds, and the inter-component API contracts — MEDS calling EAPE, SDLB and the ICAP Operator consuming EAPE outputs, MEDS consuming ICAP Operator scanning — functioned correctly across all system-level integration tests. This confirms that the modular, API-driven architecture achieves both component independence and system-level cohesion, which was the central architectural design objective.

7.3. Limitations

7.3.1. Evaluation Environment

All experiments were conducted on K3s or Minikube in local VM or workstation environments. Managed Kubernetes distributions — EKS, GKE, AKS — impose additional control plane constraints, managed networking configurations, and vendor-specific admission controller pre-configurations that have not been tested and may require component-specific adjustments. Hahn et al.'s documented security vulnerabilities in default Istio deployments suggest that SDLB's service mesh integration warrants particularly careful validation in managed environments before enterprise production deployment.

7.3.2. Weight Calibration

Both the MEDS six-factor risk weights and the EAPE seven-factor detection weights were manually calibrated against limited evaluation datasets. The MEDS weights were calibrated against 15 analytically constructed promotion scenarios; the EAPE weights were calibrated through iterative testing against representative namespace configurations. Neither weight set can claim statistical generalisability across the full diversity of real-world Kubernetes deployments, which vary substantially in naming conventions, RBAC patterns, resource quota discipline, and labelling practices. Data-driven weight optimisation approaches — gradient descent against a labelled promotion history for MEDS, or reinforcement learning optimising downstream compliance coverage for EAPE — would reduce sensitivity to the initial calibration

choices and improve robustness on cluster configurations not represented in the current evaluation sets.

7.3.3. Compliance Catalogue Scope

EAPE's compliance catalogue covers CIS Kubernetes Benchmark v1.9 (28 controls) and PCI-DSS v4.0 (16 requirements) but not SOC 2, ISO 27001, or HIPAA. The 28 CIS controls cover the namespace-level enforcement scope; the full CIS Benchmark also covers cluster configuration controls at the API server, scheduler, controller manager, and kubelet levels that are outside admission control scope. The 16 PCI-DSS requirements are those most directly applicable to Kubernetes workloads; the complete standard contains 64 requirements, some of which govern physical and organisational controls outside Kubernetes management scope.

7.3.4. Scale and Longitudinal Validity

The SDLB and ICAP Operator evaluations used three ICAP replica pods. The behaviour of the predictive autoscaling path and the adaptive routing engine under larger replica counts — five to ten instances as would be expected in a high-throughput production environment — has not been validated. The MEDS USLO model has been validated functionally but has not been evaluated through a longitudinal study of policy drift in real production deployments. A longitudinal study measuring policy drift reduction and misconfiguration incident frequency over months would provide ecological validity that controlled laboratory evaluation cannot supply.

7.4. Future Work

7.4.1. Cross-Platform Validation

Cross-platform validation on EKS, GKE, and AKS is the most immediate priority for all four components. The ClusterDetector in EAPE's detection algorithm reads cloud provider hints from node labels and taints; these require validation against the actual label conventions used by each managed distribution. SDLB's Istio integration requires validation against the managed Istio installations provided by GKE's Anthos Service Mesh and AWS App Mesh, which differ from the open-source Istio distribution used

in evaluation. The ICAP Operator's HPA custom metrics adapter requires validation against the custom metrics API implementations of each cloud provider.

7.4.2. Machine-Learning-Based Optimisation

Both the MEDS risk model and the EAPE detection algorithm are candidates for data-driven weight optimisation. The MEDS risk weights could be learned through gradient descent against a labelled dataset of promotion histories from real Kubernetes deployments, optimising for the objective of maximising the rate at which genuinely high-risk promotions are identified while minimising false positives that block legitimate low-risk changes. The EAPE detection weights could be optimised through reinforcement learning against a compliance coverage objective, learning which namespace structural properties most reliably predict environment tier across diverse cluster configurations. Both approaches would address the most significant limitation of the current manually calibrated models.

7.4.3. Extended Compliance Frameworks

Extending EAPE's compliance catalogue to SOC 2, ISO 27001, and HIPAA would significantly broaden the platform's applicability in regulated industries. The NIST SP 800-190 risk taxonomy already used as the organisational backbone of the CIS and PCI-DSS mappings provides a natural bridge to these additional frameworks: each framework's requirements can be mapped to one of the five NIST risk areas and then to the corresponding Kubernetes admission control constraint. The ISO 27001 Annex A control set, in particular, overlaps substantially with the CIS Benchmark controls already in the EAPE catalogue, reducing the incremental authoring effort required for full ISO 27001 coverage.

7.4.4. Continuous Integration and Platform Integration

Integration of MEDS's deployment history with EAPE's continuous change detection would enable end-to-end audit trails linking every promotion event to subsequent policy classification changes, providing a complete compliance evidence chain for regulatory reporting. The ICAP Operator's predictive autoscaling path could be extended with a machine-learning-based health score forecasting model that learns the

cluster's workload patterns over time — for example, recognising daily traffic peaks and pre-emptively scaling inspection capacity before the peak arrives, rather than reacting to the health score decline that accompanies it. SDLB's adaptive decision engine could similarly benefit from a pattern-learning extension that identifies recurring traffic spike profiles and transitions to spread mode pre-emptively.

7.4.5. Longitudinal Production Study

A longitudinal study of CAPSLock's security posture improvements in a real production deployment would provide the ecological validity that controlled laboratory evaluation cannot. Measuring policy drift reduction over months, misconfiguration incident frequency before and after deployment, and compliance audit preparation time would quantify the operational value of the platform in terms directly meaningful to security operations teams considering adoption. Such a study would also surface the edge cases and operational failure modes that only emerge under the diversity of real production workloads, informing the next iteration of each component's design.

REFERENCES

[1] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study," ACM Trans. Softw. Eng. Methodol., vol. 32, no. 4, Art. 99, Jul. 2023, doi: 10.1145/3579639.

[2] M. S. I. Shamim, F. A. Bhuiyan, and A. Rahman, "XI Commandments of Kubernetes Security," in Proc. IEEE SecDev, Atlanta, GA, 2020, pp. 58–64, doi: 10.1109/SecDev45635.2020.00025.

[3] S. Sultan, I. Ahmad, and T. Dimitriou, "Container Security: Issues, Challenges, and the Road Ahead," IEEE Access, vol. 7, pp. 52976–52996, 2019, doi: 10.1109/ACCESS.2019.2911732.

[4] F. Minna, A. Blaise, F. Rebecchi, B. Chandrasekaran, and F. Massacci, "Understanding the Security Implications of Kubernetes Networking," IEEE Security & Privacy, vol. 19, no. 5, pp. 46–56, 2021.

[5] C. Cesarano and R. Natella, "KubeFence: Security Hardening of the Kubernetes Attack Surface," in Proc. 55th IEEE/IFIP DSN, 2025, doi: 10.1109/DSN64029.2025.00054.

[6] Open Policy Agent Project, OPA: Policy-based Control for Cloud Native Environments, CNCF, 2021. [Online]. Available: https://www.openpolicyagent.org

[7] Kyverno Project, Kyverno: Kubernetes Native Policy Management, CNCF, 2021. [Online]. Available: https://kyverno.io

[8] A. Sissodiya et al., "Formal Verification for Preventing Misconfigured Access Policies in Kubernetes Clusters," IEEE Access, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3597504.

[9] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, "Exploring Security Practices in Infrastructure as Code," Empirical Softw. Eng., vol. 29, Art. 182, 2024, doi: 10.1007/s10664-024-10610-0.

[10] L. Rice, Container Security: Fundamental Technology Concepts that Protect Containerized Applications, O'Reilly Media, 2020, ISBN: 978-1492056706.

[11] J. Elson and A. Cerpa, Internet Content Adaptation Protocol (ICAP), IETF RFC 3507, Apr. 2003. Available: https://datatracker.ietf.org/doc/html/rfc3507

[12] W. Li, Y. Lemieux, J. Gao, Z. Zhao, and Y. Han, "Service Mesh: Challenges, State of the Art, and Future Research Opportunities," in Proc. IEEE SOSE, 2019, pp. 122–127.

[13] M. R. Saleh Sedghpour, C. Klein, and J. Tordsson, "An Empirical Study of Service Mesh Traffic Management Policies for Microservices," in Proc. ACM/SPEC ICPE, 2022, doi: 10.1145/3489525.3511686.

[14] R. Chandramouli and Z. Butcher, Building Secure Microservices-based Applications Using Service-Mesh Architecture, NIST SP 800-204A, NIST, 2020, doi: 10.6028/NIST.SP.800-204A.

[15] D. A. Hahn, D. Davidson, and A. G. Bardas, "MisMesh: Security Issues and Challenges in Service Meshes," in SecureComm 2020, LNICST vol. 335, Springer, 2020.

[16] T.-T. Nguyen, Y.-J. Yeom, T. Kim, D.-H. Park, and S. Kim, "Horizontal Pod Autoscaling in Kubernetes for Elastic Container Orchestration," Sensors, vol. 20, no. 16, Art. 4621, 2020, doi: 10.3390/s20164621.

[17] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," Commun. ACM, vol. 59, no. 5, pp. 50–57, 2016.

[18] Center for Internet Security, CIS Kubernetes Benchmark v1.9, CIS, 2024. Available: https://www.cisecurity.org/benchmark/kubernetes

[19] PCI Security Standards Council, PCI DSS v4.0, PCI SSC, 2022. Available: https://www.pcisecuritystandards.org

[20] M. Souppaya, J. Morello, and K. Scarfone, Application Container Security Guide, NIST SP 800-190, NIST, 2017, doi: 10.6028/NIST.SP.800-190.

[21] M. Hausenblas and S. Schimanski, Programming Kubernetes, O’Reilly Media, 2019, ISBN: 978-1492047100.

[22] J. Dobies and J. Wood, Kubernetes Operators, O’Reilly Media, 2020, ISBN: 978-1492048046.

[23] CNCF TAG Security, Cloud Native Security Whitepaper v2, CNCF, 2022. Available: https://www.cncf.io/reports/cloud-native-security-whitepaper

[24] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, Zero Trust Architecture, NIST SP 800-207, NIST, 2020, doi: 10.6028/NIST.SP.800-207.

[25] D. D'Silva and D. Ambawade, "Building A Zero Trust Architecture Using Kubernetes," in Proc. IEEE I2CT, 2021.